



**POLITECNICO
MILANO 1863**

**062249 - Vehicle Optimal Design
Mechanical Engineering
Automotive and Motorsport Engineering, M.Sc.**

Laboratories Summary

Group:

Pietro Nardon
Gianluca Meli

December 17, 2024

Contents

1	<u>Lab.1 Braking distribution</u>	1
1.1	Problem definition	1
1.2	Optimization problem	3
1.3	Results	4
1.3.1	Pistons diameters	4
1.3.2	Results dependence on initialization point	4
1.3.3	Problem reformulation	6
2	<u>Lab.2 Multi-objective optimization quarter car model</u>	7
2.1	Problem definition	7
2.2	Optimization problem	7
2.3	Results	9
2.3.1	Objective functions space evaluation	9
2.3.2	Pareto-optimal set evaluation: Road holding + Discomfort	10
2.3.3	Pareto-optimal set evaluation: Road holding + Discomfort + Working space	10
3	<u>Lab.3 Pareto-optimal set refinement quarter car model</u>	13
3.1	Optimization problem	13
3.1.1	Weighted sum method	13
3.1.2	ε -constrain method	14
3.1.3	Analytical formulation	15
3.2	Results	16
3.2.1	Uniform grid method	16
3.2.2	Weighted sum method	16
3.2.3	ε -constrain method	17
3.2.4	Analytical formulation	18
3.2.5	General remarks	19
4	<u>Lab.4 Pareto-optimal set active suspension</u>	20
4.1	Problem definition	20
4.2	Optimization problem	21
4.2.1	Uniform grid method	21
4.2.2	ε -constrain method	21
4.3	Passive/Active suspensions comparison	22
5	<u>Lab.5 Genetic Algorithm</u>	23
5.1	Problem definition	23
5.2	Results	24
5.2.1	No mutation, no selection	24
5.2.2	Mutation, no selection	24
5.2.3	Mutation, selection	25
6	<u>Lab.6 Double wishbone suspension design with GA</u>	26
6.1	Problem definition	26
6.2	Optimization problem	27

6.3	Genetic Algorithm	27
6.3.1	Evaluation	27
6.3.2	Cycle	28
6.4	Results	29
6.4.1	Number of bits: 10 Mutation probability: 2.23 %	30
6.4.2	Number of bits: 5 Mutation probability: 2.12 %	31
6.4.3	Number of bits: 20 Mutation probability: 2.18 %	32
6.4.4	Number of bits: 5 Mutation probability: 10.00 %	33
6.4.5	Number of bits: 10 Mutation probability: 0.16 %	34
6.4.6	Number of bits: 16 Mutation probability: 1.58 % 50x50 DV linspace	35
6.4.7	Number of bits: 16 Mutation probability: 1.58 % 50x50 DV rand	36
7	Lab.7 Neural Network creation and training	37
7.1	Problem definition	37
7.2	Training example results	38
7.2.1	Noisy sine	38
7.2.2	Exponential function	39
8	Lab.8 - Multi-objective handling optimization	42
8.1	Problem definition	42
8.2	Requests and results	44
8.2.1	Requests 1 and 2 - Spearman Correlation	44
8.2.2	Requests 3 and 4 - Neural Network training	45
8.2.3	Request 5 - Pareto front	52
8.3	Addendum - <i>VI-CarRealTime</i>	53
8.3.1	Solution 1 - Minimum yaw rate variation	53
8.3.2	Solution 2 - Maximum lateral acceleration	54
8.3.3	Solution 3 - Minimum curvature	54
9	Lab.9 - Structural optimization	56
9.1	Problem definition	56
9.2	99 line topology optimization code	57
9.2.1	Function definition	57
9.2.2	Request 1: Mesh independency	58
9.2.3	Request 2: Penalization effects	59
9.3	Altair Inspire	60
9.3.1	Problem setup	60
9.3.2	Results	61

1 Lab.1 Braking distribution

1.1 Problem definition

The main objective of this laboratory was to find the ideal braking distribution in a commercial vehicle using a non linear minimisation process, in order to find the ideal diameters that the front and rear brake disk should have to maximise the braking force.

Starting from the equilibrium equations of a braking vehicle, the model was formulated and the ideal braking forces were calculated. The model was developed on the bases of the Saab 9-2X 2.0i 16V T Aero (230 hp) Petrol (Fig. 1) for which the useful data are shown in (Tab. 1).



Figure 1: Saab 9-2X

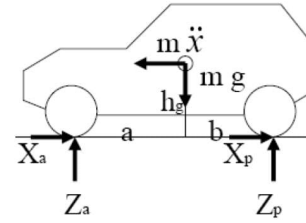


Figure 2: Vehicle equilibrium

To develop the model a vertical and horizontal forces and a moment (about the front wheel contact point) equilibrium were considered, as shown in (Eq. 1.1) and (Fig. 2).

$$\begin{cases} Z_A + Z_P = mg \\ X_A + X_P = m\ddot{x} \\ m\ddot{x}h_g - mga + Z_P(a+b) = 0 \end{cases} \quad (1.1)$$

The maximum longitudinal braking force were estimated by considering the maximum force that the tyre can exchange with the ground, using the Coulomb's friction force model (Eq. 1.2).

$$\begin{cases} X_{A,max} = \mu_{tyre} Z_A \\ X_{P,max} = \mu_{tyre} Z_P \end{cases} \quad where \quad \begin{cases} Z_A = mg \left(\frac{b}{a+b} + \frac{\ddot{x}}{g} \frac{h_G}{a+b} \right) \\ Z_P = mg \left(\frac{a}{a+b} - \frac{\ddot{x}}{g} \frac{h_G}{a+b} \right) \end{cases} \quad (1.2)$$

Then combining (Eq. 1.1) and (Eq. 1.2) it was possible to find the ideal braking forces, as in (Eq. 1.3).

$$\begin{cases} X_{A,max} = m\ddot{x} \left(\frac{b}{a+b} + \frac{\ddot{x}}{g} \frac{h_G}{a+b} \right) \\ X_{P,max} = m\ddot{x} \left(\frac{a}{a+b} - \frac{\ddot{x}}{g} \frac{h_G}{a+b} \right) \end{cases} \quad (1.3)$$

To find the actual braking distribution it is needed to compute the braking force of the pistons acting on the disk of a single wheel. After computing the single wheel force, by assuming that front and rear wheel have same number of pistons $N_{pistons}$, same friction coefficient μ_{disk} and same effective radius r/R , is finally possible to compute the real braking force distribution as shown in (Eq. 1.4), where p is the hydraulic pressure of the braking system.

$$X = \frac{r}{R} \mu_{disk} p \frac{\pi d^2}{4} 2N_{pistons} \longrightarrow X_P = X_A \frac{A_P}{A_A} \quad (1.4)$$

The brake distribution curve was evaluated over the acceleration, starting from 0 m/s up to the maximum possible acceleration before slipping, which is μg , using as data ¹ for the calculations (Tab. 1) and initial diameter $d_A = 0.03 \text{ m}$ and $d_P = 0.02 \text{ m}$. As can be seen in (Fig. 3), over a certain value $X_{P,real}$ is greater than the ideal curve (obtained with the aim of letting the front wheels to lock before the rear ones for stability issues), which means that the real wheels will lock making the vehicle unstable. To tackle this problem, the braking pressure of the rear brakes is limited by a splitter that keeps the pressure at the rear brakes constant, which is identified by the yellow line in (Fig. 3).

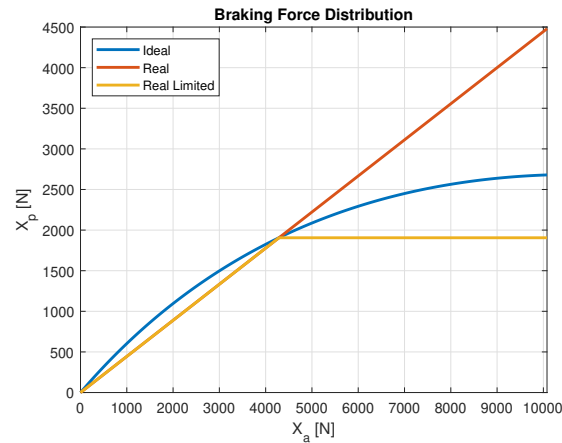


Figure 3: Brake distribution

Vehicle				Brake	
l	2.525 m	m	1445 kg	μ_{disk}	0.4
a	1.010 m	μ_{tyre}	0.9	r/R	0.5
b	1.515 m	g	9.81 m/s	$N_{pistons}$	2
h_G	0.533 m				

Table 1: Vehicle and Brake data

¹<https://www.auto-data.net/en/saab-9-2x-2.0-i-16v-t-aero-230hp-11839>

1.2 Optimization problem

In order to maximize the performance of the real braking system, its configuration should be designed to minimize the difference that it has respect to the ideal distribution.

An optimization problem, varying the real braking forces, was set up to find the best possible configuration. As design variables of the problem, it was decided to use the pistons diameters of the front and rear axle respectively, since they are the easiest parameter to change in the braking system. Both diameters were constrained between 15 mm and 50 mm . The studied objective function $E(\underline{d})$ is the mean squared error between the ideal and the real distribution curves, which depends on the pistons diameters; shown in (Eq. 1.5), where N is the number of points used to represent the curves.

$$E(\underline{d}) = \frac{1}{N} \sum (X_{P,max} - X_P(\underline{d}))^2 \quad (1.5)$$

This resulted in the optimization problem stated in (Eq. 1.6), where the objective function $E(\underline{d})$ was minimized to find the optimal diameters, while respecting the constraints.

$$\begin{cases} \min(E(\underline{d})) = \min(\frac{1}{N} \sum (X_{P,max} - X_P(\underline{d}))^2) \\ \underline{d} = [d_A \quad d_P] \in \Omega = [15 \quad 50] \times [50 \quad 15] \end{cases} \quad (1.6)$$

To solve the problem on MATLAB a function handle computing the error between the ideal and the real contribution was created and used as input inside the function `fminsearch`, together with a set of initial conditions $\underline{d}_0 = [30\text{ mm} \quad 20\text{ mm}]$.

1.3 Results

1.3.1 Pistons diameters

After solving the minimization problem it was possible to find the optimal pistons diameter starting from the given initial conditions. The results can be seen in (Tab. 2).

	$\underline{d}_0$	$\underline{d}_{opt}$
d_A	30 mm	31.10 mm
d_P	20 mm	19.18 mm

Table 2: Optimal parameter

The optimal distribution curve, shown in yellow in (Fig. 4), is the one that minimizes the difference respect to the ideal one and is plotted using the optimal diameters found trough the optimization problem.

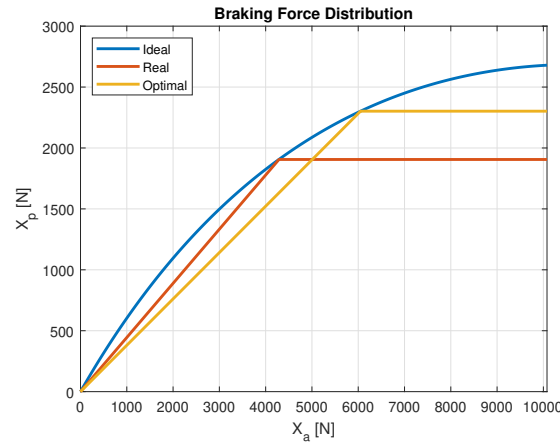


Figure 4: Optimal distribution

1.3.2 Results dependence on initialization point

It can be noted that, by changing the values of the initial diameters used to initialize the optimization problem, the solution is influenced. In (Tab. 3) different solutions of the problem using different initial conditions $\underline{d}_0$ are reported and straightaway is easy to spot the difference between each optimal diameter vector $\underline{d}_{opt}$, indicating a clear dependence of the solution on the initial conditions.

	$\underline{d}_{0,1}$	$\underline{d}_{opt,1}$	$\underline{d}_{0,2}$	$\underline{d}_{opt,2}$	$\underline{d}_{0,3}$	$\underline{d}_{opt,3}$	$\underline{d}_{0,4}$	$\underline{d}_{opt,4}$
d_A	30 mm	31.09 mm	50 mm	31.01 mm	40 mm	44.05 mm	25 mm	28.23 mm
d_P	20 mm	19.18 mm	15 mm	19.17 mm	30 mm	27.19 mm	20 mm	17.39 mm
d_A/d_P	1.5	1.621	3.333	1.618	1.333	1.619	1.25	1.619

Table 3: Initial condition dependence

By analysing closely the problem a pattern between the solutions can be found, in particular, it can be noted that the optimization problem is not minimizing the objective function by directly changing the front and rear pistons diameters but it's optimizing the ratio between the two. Even if the optimal diameters are very different depending on the initial condition, their ratio is relatively stable being $d_A/d_P \approx 1.62$, as shown in (Tab. 3). This behaviour is again confirmed by the plots in (Fig. 5): the optimal curve, colored in yellow, is approximately always the same, even if the found optimal diameters were very different between each case. As said before, this behaviour is due to the problem minimizing the object function trough the optimal ratio d_A/d_P instead of the single diameters.

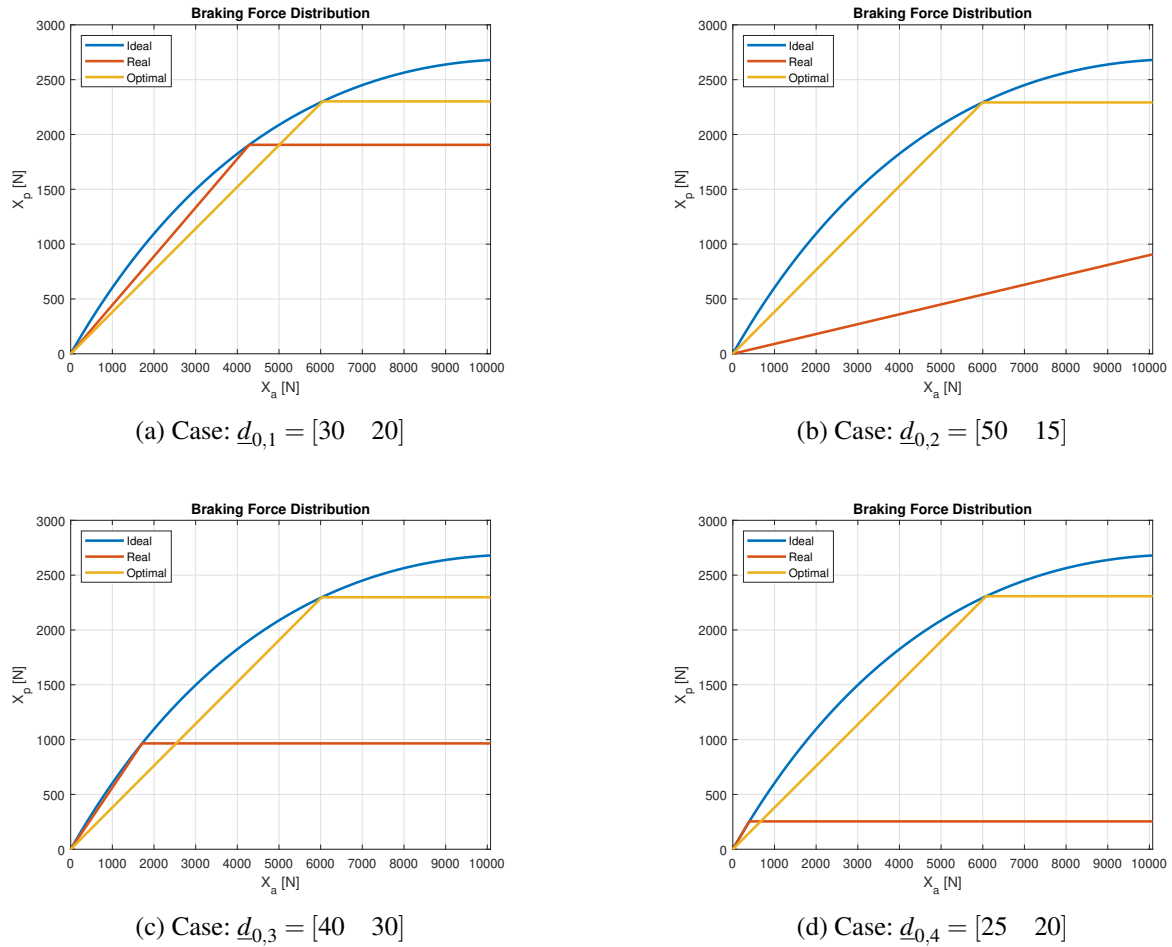


Figure 5: Influence of initial condition

This behaviour can be explained analysing (Eq. 1.4): X_P is dependent on X_A and on the pistons areas A_P and A_A but assuming that both pistons have a circular section the equation can be rewritten in (Eq. 1.7). Here the dependence of X_P on the diameter ratio can be seen and demonstrated.

$$X_P = X_A \frac{A_P}{A_A} = X_A \frac{\cancel{\pi} d_P^2}{\cancel{\pi} d_A^2} = X_A \left(\frac{d_P}{d_A} \right)^2 \quad (1.7)$$

1.3.3 Problem reformulation

It can be observed that this optimization problem is not completely well posed.

In particular, it can be noticed that modifying the initial guess values leads to different pistons diameter optimal dimensions. What is constant instead is the ratio between the two values, which in fact represent the only one solution reached by the optimization process. This ratio is the only optimization variable that could be used for this kind of problem.

This kind of behavior suggests the presence of multiple solutions (i.e. multiple global minima) in our optimization function. To verify this, we plotted in (Fig. 6) all the values of the MSE function $E(\underline{d})$ corresponding to each combination of front and rear pistons diameters.

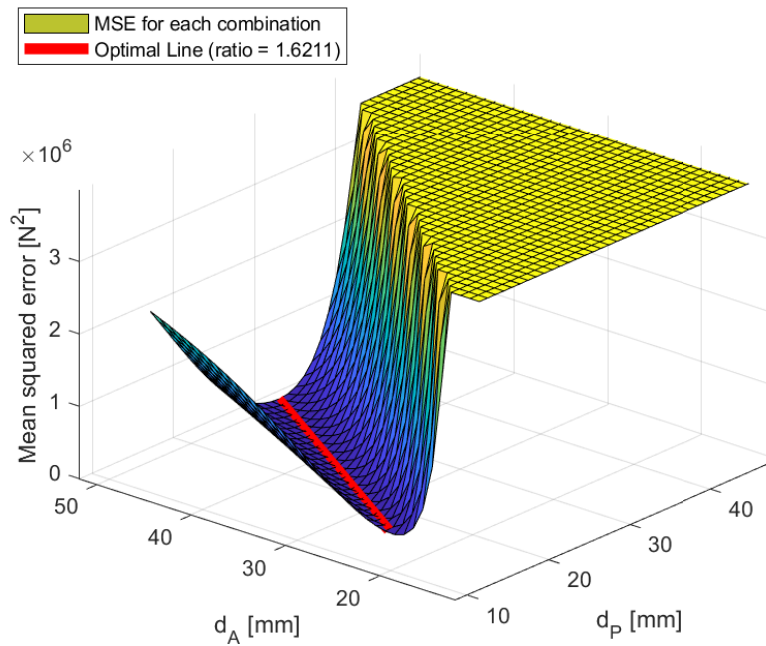


Figure 6: Mean Squared Error

From this 3D diagram it's clear how the MSE reaches its minima values along a straight line, confirming the hypothesis: our Objective Function is not strictly convex (in particular, given a minimum, the other values are greater **or equal** to it). Moreover, by plotting a line (red line in (Fig. 6)) which corresponds to all the feasible values for which $d_A/d_P = d_{ratio}^{opt}$, it can be observed that it correspond exactly to the line of the MSE minima, confirming our optimization results.

Another observation regards the yellow area: it corresponds to those values for which the slope of the real line is greater than the initial slope of the real brake distribution, which leads to a constantly null real brake distribution, that makes the error to be the maximum possible and equal to the integral of the ideal distribution (i.e. the subtended area).

Finally, a well posed problem may be obtained by considering a single variable optimization problem, using as variable the ratio between the two areas instead of the two single diameters. The correct values of the diameters could be then obtained by combining the information about the required braking force and the desired pressure inside the braking system.

2 Lab.2 Multi-objective optimization quarter car model

2.1 Problem definition

The scope of the analysis was to study the dynamic response of a vehicle subjected to an external excitation given by road irregularities. The analysis was made to find the optimal suspension which allows for a smooth ride and, at the same time, the capability to keep a as much as stable contact with the road. To simplify the problem a quarter car vehicle model was used to model the suspension behaviour and its relation with the tyre and the vehicle mass. The model is shown in (Fig. 7), while the value of the parameters are displayed in (Tab. 4). The input of the system is road unevenness.

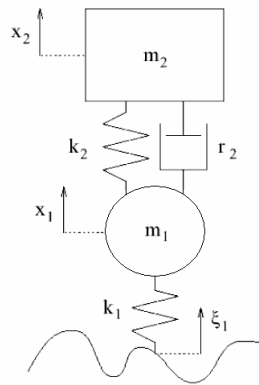


Figure 7: Quarter car vehicle model

Parameter	Value	Design variable	Field of variation
A_b	$1.4 \cdot 10^{-5} m$	k_2	$0 : 50000 N/m$
v	$30 m/s$	r_2	$0 : 5000 Ns/m$
m_1	$230 kg$		
m_2	$30 kg$		
k_1	$12000 N/m$		

Table 4: Suspension model data

2.2 Optimization problem

In order to maximise the effectiveness of the suspension, an optimization problem was set up to find the best possible set of suspension parameters.

The objective functions to be minimized were: discomfort (Eq. 2.1), road holding (Eq. 2.2) and working space (Eq. 2.3). The suspension equivalent stiffness k_2 and damping r_2 were chosen as field variable, being the values that more influence the vehicle behaviour while, at the same time, being relatively easy to change in a real life suspension. Discomfort, road holding and working space are described as standard deviation of the vehicle vertical acceleration $\ddot{x}_2$, vertical force on the ground F_z and relative displacement $x_2 - x_1$ between the masses respectively. The standard deviation is used since, assuming to describe the

road unevenness through a Gaussian distribution, it is possible to estimate and analyse the peaks of each studied quantity and the variation respect to a mean value. The parameter A (Eq. 2.4) is used to account for the road unevenness, using its power spectral density.

$$\sigma_{\ddot{x}_2} = A \cdot \sqrt{\frac{m_1 + m_2}{m_2^2 r_2} k_2^2 + \frac{k_1 r_2}{m_2^2}} \quad (2.1)$$

$$\sigma_{F_z} = A \cdot \sqrt{\frac{(m_1 + m_2)^3}{m_2^2 r_2} k_2^2 - 2 \frac{m_1 k_1 (m_1 + m_2)}{m_2 r_2} k_2 + \frac{k_1 r_2 (m_1 + m_2)^2}{m_2^2} + \frac{k_1^2 m_1}{r_2}} \quad (2.2)$$

$$\sigma_{x_2 - x_1} = A \cdot \sqrt{\frac{m_1 + m_2}{r_2}} \quad (2.3)$$

$$A = \sqrt{\frac{A_b v}{2}} \quad (2.4)$$

All the following plots will have the road holding variable limited to $900N$, which is approximately the static weight of the vehicle. As the road holding is defined as a standard deviation around its static weight, most of the results will be around it, moreover designing a vehicle with higher road holding is not feasible, since the car would leave the ground.

2.3 Results

2.3.1 Objective functions space evaluation

Let us consider, at first, the case of two Objective Functions (namely Discomfort and Road Holding).

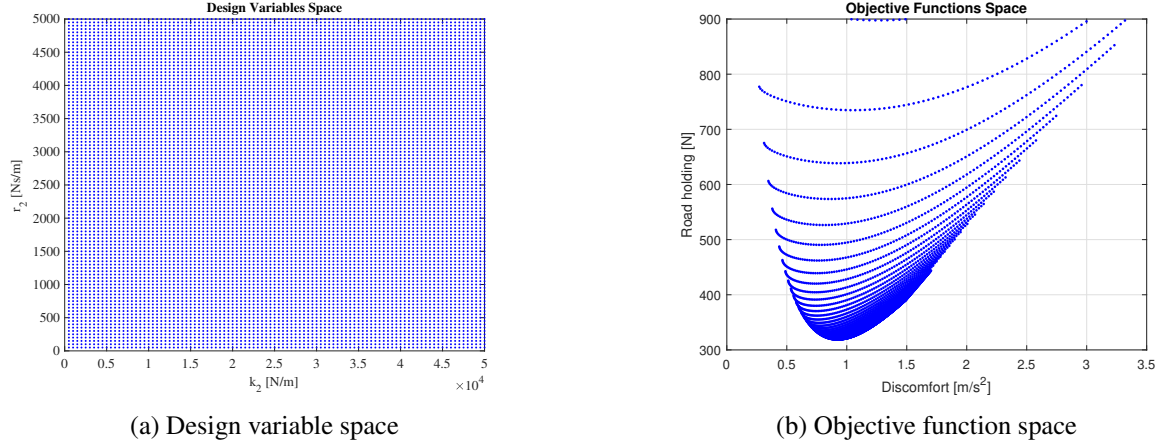


Figure 8: Design Variables and Objective Functions spaces evaluation

By looking at (Fig. 8) we can observe the chosen Design Variables Space which ranges in the intervals $k_2 = [0, 50000] \text{ N/m}$ and $r_2 = [0, 5000] \text{ Ns/m}$, and the consequent Objective Functions space, found by evaluating the equations presented in (Sec. 2.2) for all the different possible combinations of k_2 and r_2 .

From the literature is known that the Pareto Optimal Set is defined by the "lower-left" boundary of the OF Space, so we expect to find the Optimal Solutions along that border, as shown in (Sec. 2.3.2).

By considering instead the case of three Objective Functions (namely Discomfort, Road Holding and Working Space), the resulting space is shown in (Fig. 9)

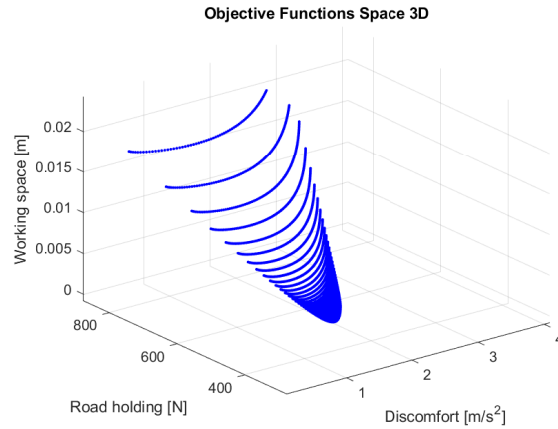


Figure 9: 3-OFs Space

Similarly to the case of two Objective Functions, we expect the Pareto Set to be a boundary surface placed facing towards the origin of the axes (i.e. the surface which minimize all the three functions at the same time), as shown in (Sec. 2.3.3).

2.3.2 Pareto-optimal set evaluation: Road holding + Discomfort

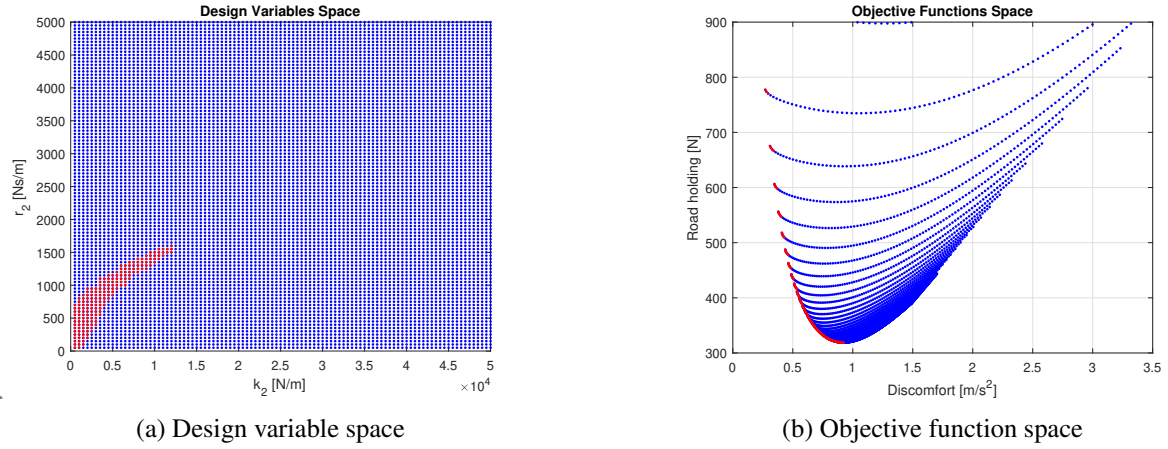


Figure 10: Optimization problem using two objective functions

As previously commented in (Sec. 2.3.1), the Pareto Optimal Set, for a 2-OFs problem, is positioned on the "lower-left" boundary of the Objective Functions space. Regarding the Design Variables space, we obtained the shape that was suggested by the literature for this kind of problem (in terms of DVs and OFs relations).

These two observations make us trust the achieved results.

2.3.3 Pareto-optimal set evaluation: Road holding + Discomfort + Working space

The main issue in this case is how to reformulate the problem, extending the algorithm adopted for the 2D case to a 3D one. To achieve that, we traduced the exclusion-from-Pareto condition in a code by applying the definition of a Pareto solution:

x_i is a Pareto-optimal solution whenever $\nexists$ a solution x_j such that:

$$\begin{cases} f_k(x_j) \leq f_k(x_i) & \text{with } k = 1, 2, 3, \dots, n_{of} \\ \exists l : f_l(x_j) < f_l(x_i) \end{cases}$$

meaning that there are no other solutions for the which you can make better one OF without worsening at least another one.

This condition is translated into a Matlab code considering the "opposite" situation, that is after having pre-allocated our entire space as all Pareto solutions, we consider a solution not belonging to the Pareto set if $\exists$ at least one other solution for which you can make better one OF without worsening at least another one.

In particular:

```

for ii = 1:size(DV_Mat_3D, 1)
    for jj = 1:size(DV_Mat_3D, 1)
        if all([DV_Mat_3D(jj,3) <= DV_Mat_3D(ii,3),...
                DV_Mat_3D(jj,4) <= DV_Mat_3D(ii,4),...
                DV_Mat_3D(jj,5) <= DV_Mat_3D(ii,5)])...
            &&...
            any([DV_Mat_3D(jj,3) < DV_Mat_3D(ii,3),...
                DV_Mat_3D(jj,4) < DV_Mat_3D(ii,4),...
                DV_Mat_3D(jj,5) < DV_Mat_3D(ii,5)])

                DV_Mat_3D(ii,6) = 1;
                break;
            end
        end
    end
end

```

The results are plotted in (Fig. 11).

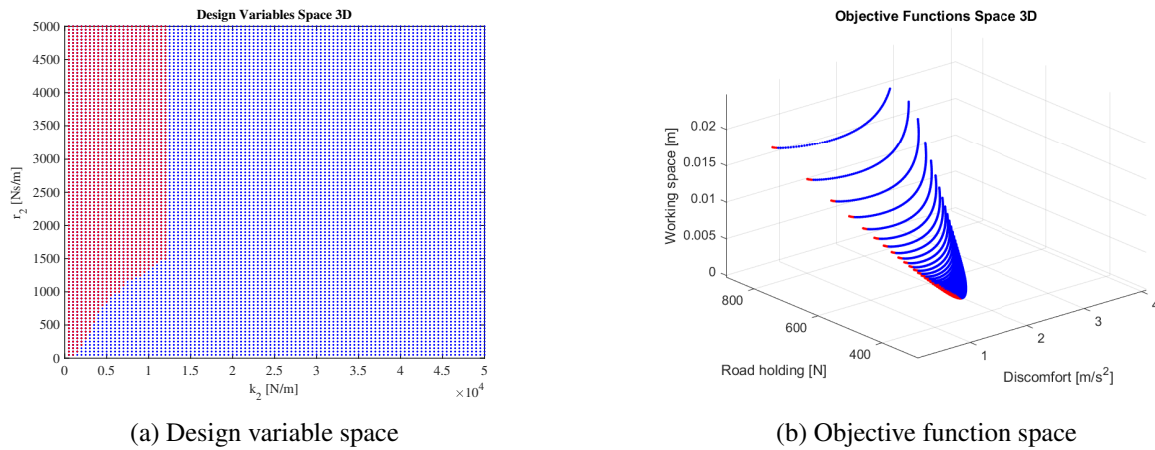
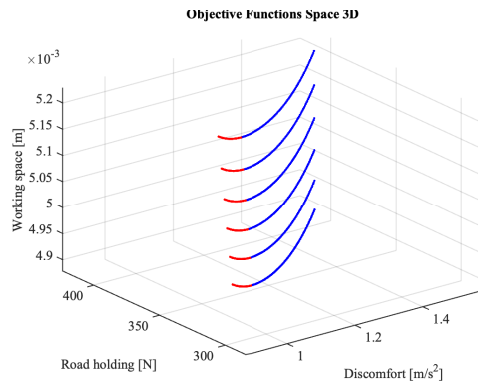


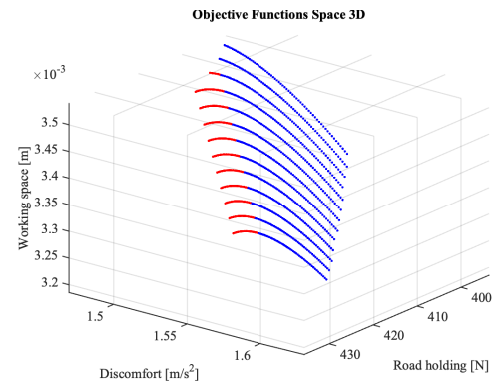
Figure 11: Optimization problem using three objective functions

As previously commented in (Sec. 2.3.1), the Pareto Optimal Set, for a 3-OFs problem, is constituted by a boundary surface of the Objective Functions space, facing the origin of the 3D diagram, which makes us confident in the achieved results.

In particular, the two following zoomed views of the Pareto set (Fig. 12), better show this expected shape.



(a) Detail of the side part of the Pareto set



(b) Detail of the bottom part of the Pareto set

Figure 12: Zoomed views of the Pareto set

3 Lab.3 Pareto-optimal set refinement quarter car model

An effort was made in this laboratory to increase the precision and the density of the Pareto front in the problem described in (Sec. 2.1). In particular, it was analyzed the optimization problem using as objective function discomfort (Eq. 2.1) and road holding (Eq. 2.2), with design variables k_2 and r_2 again.

3.1 Optimization problem

As specified before the optimization problem is the same described in (Sec. 2.2), focusing on the case with only two objective functions. The main difference with the previous problem is in the way the Pareto optimal front is computed. In (Sec. 2.2) a more rigorous definition, paired with a uniform distribution of the design variable, was used to compute the front, but the results were highly dependent on the grid resolution and some points that were actually Pareto were left out as a consequence. More precise methods have been implemented in this laboratory to find an improved way to compute the Pareto front.

3.1.1 Weighted sum method

The main idea here is to minimize a scalar single objective function, which represents the weighted sum of the OFs, instead of all the OFs themselves, using different weighting factors to move along the entire Pareto front and effectively tackling the multi objective minimization as a single objective one. All the solutions coming from the minimization of the scalar problem (Eq. 3.1) will be non-dominated solutions for the original multi-objective space. In our case the problem will be as follows.

$$\begin{cases} \min_{\mathbf{x} \in \mathcal{F}} \phi(k_2, r_2) \\ \phi(k_2, r_2) = \lambda_1 \frac{\sigma_{\ddot{x}_2}(k_2, r_2)}{\max(\sigma_{\ddot{x}_2})} + \lambda_2 \frac{\sigma_{F_z}(k_2, r_2)}{\max(\sigma_{F_z})} \\ \lambda_1 + \lambda_2 = 1 \end{cases} \quad (3.1)$$

The main issue of this method is tackling problems with non-convex OFs space: the weighted sum algorithm cannot find the whole Pareto front if it is used in such types of problems. The method was implemented in Matlab via a for loop ranging from the minimum to maximum possible value of λ .

```
x0 = [0, 0];
ind = 0;
lambda_vect = 0:0.001:1;
Pareto = zeros(length(lambda_vect), 4);

for lambda = lambda_vect
    phi = @(x) lambda*disc_fun_DV(x(1), x(2))/m_fob1 + (1-lambda)*
        RH_fun_DV(x(1), x(2))/m_fob2;
    ind = ind + 1;
    x_opt = fminsearch(phi, x0);
    Pareto(ind, :) = [x_opt, disc_fun_DV(x_opt(1), x_opt(2)),
        RH_fun_DV(x_opt(1), x_opt(2))];
end
```

3.1.2 ε -constrain method

This method is based on minimizing only one objective function, while the others are taken into account through some moving constrains, in this way a multi-objective problem is transformed into a single objective one. By iteratively decreasing the values of the constrains, sweeping the objective function space entirely, and solving the problem each time, it is possible to compute the Pareto optimal front, even of a non-convex problem. The quarter car problem has been reformulated by considering the road holding σ_{F_z} as the function to be minimized, while the discomfort $\sigma_{\ddot{x}_2}$ is transformed into a moving constraint, as follows.

$$\begin{cases} \min_{\mathbf{x} \in \mathcal{F}} \sigma_{F_z}(k_2, r_2) \\ \sigma_{\ddot{x}_2}(k_2, r_2) \leq \varepsilon \end{cases} \quad (3.2)$$

Inside the for loop in the Matlab implementation, the values chosen for ε , to implement the moving constraint, were the values of discomfort $\sigma_{\ddot{x}_2}$ which were close to the beginning of the Pareto front along the axis where $\sigma_{\ddot{x}_2}$ was plotted. The computations were carried out both using the "sqp" method, which approximates the Hessian matrix, and the "interior-point" method and the results were then compared.

```
RH_fun_constr = @(x) A*sqrt(((m_1 + m_2)^3./(m_2^2 .* x(2)))).*x(1)
    .^2 - 2*((m_1*k_1*(m_1 + m_2))./(m_2.*x(2))).*x(1) + (k_1.*x(2)
    *(m_1 + m_2)^2)/(m_2^2) + (k_1^2*m_1)./x(2));

ind = 0;
x0_constr = [20, 20];
lb = [10, 10]; % [lb_k2, lb_r2]
ub = [50000, 5000]; % [ub_k2, ub_r2]
options = optimset('Algorithm', 'sqp', 'Display', 'off');
for eps = 1.5:-0.005:0
    ind = ind + 1;
    nonlinconstr = @(x) non_lin_constr(x, A, m_1, m_2, k_1, eps);
    x_opt_constr = fmincon(RH_fun_constr, x0_constr, [], [], [],
        [], lb, ub, nonlinconstr, options);
    Pareto_constr(ind, :) = [x_opt_constr, disc_fun_DV(
        x_opt_constr(1), x_opt_constr(2)), RH_fun_DV(x_opt_constr(1)
        , x_opt_constr(2))];
end
```

```
function [c, ceq] = non_lin_constr(x, A, m_1, m_2, k_1, eps)
c = A*sqrt(((m_1 + m_2)./(m_2^2 .* x(2)))).*x(1).^2 + (k_1.*x(2))/
    m_2^2) - eps;
ceq = [];
end
```

3.1.3 Analytical formulation

Using the Fritz John optimality condition, it is possible to find an analytical formulation of the Pareto front of this simple unconstrained problem. Through a relaxed version of the FJ condition, the gradients of the OFs are imposed to have the same direction, but no condition is applied on their versus. Therefore it becomes just a necessary condition and, by solving the (Eq. 3.3) for one of the variables, an analytical formulation of the type $k_2 = f(r_2)$ is found, which is a function containing the Pareto optimal set in the design variable domain, together with some points which are not Pareto. In order to correctly use this conditions the minima of each OF must be found, since the real Pareto front will be exclusively the part of the function contained between them.

Subsequently, it is possible to evaluate and plot the optimal values found solving (Eq. 3.3).

$$\frac{\partial \sigma_{\ddot{x}_2}}{\partial k_2} \cdot \frac{\partial \sigma_{F_z}}{\partial r_2} = \frac{\partial \sigma_{\ddot{x}_2}}{\partial r_2} \cdot \frac{\partial \sigma_{F_z}}{\partial k_2} \quad (3.3)$$

To implement this method in Matlab, the use of the symbolic toolbox is needed, to compute the derivatives and solutions analytically. The objective functions were evaluated using the points created to show the Pareto front in the design variable domain.

```
Dis_sym(k2,r2) = A*sqrt(((m_1 + m_2)./(m_2^2 .* r2)).*k2.^2 + (k_1
.*r2)/m_2^2);
RH_sym(k2,r2) = A*sqrt(((m_1 + m_2)^3./(m_2^2 .* r2)).*k2.^2 -
2*((m_1*k_1*(m_1 + m_2))./(m_2.*r2)).*k2 + (k_1.*r2*(m_1 + m_2)
^2)/(m_2^2) + (k_1^2*m_1)./r2);

dDis_dk2 = diff(Dis_sym,k2);
dDis_dr2 = diff(Dis_sym,r2);
dRH_dk2 = diff(RH_sym,k2);
dRH_dr2 = diff(RH_sym,r2);

grad_RH = simplify(gradient(RH_sym));
min_RH_DV = solve(grad_RH == 0);
grad_dis = simplify(gradient(Dis_sym));
min_dis_DV = solve(grad_dis == 0);

FJcon = dDis_dk2*dRH_dr2 - dDis_dr2*dRH_dk2;
FJconRES = solve(FJcon,k2);

figure(7)
[r2_anal, k2_anal] = fplot(FJconRES(1,1),[0 2750]);
fplot(FJconRES(1,1),[0 2750],'.r')

Dis_anal_sol = disc_fun_DV(k2_anal,r2_anal);
RH_anal_sol = RH_fun_DV(k2_anal,r2_anal);
```

3.2 Results

3.2.1 Uniform grid method

The uniform grid is a stable method, which always gives a solution, though with a high computational time. Unfortunately, as shown in (Sec. 2.3), the solution is not precise and can be regarded as a region where the real Pareto front will be, being this region sharper while the samples in the DV grid densify. This makes the uniform grid method a solid first try method, useful to tune the parameters of more precise methods, as shown in the subsequent sections.

3.2.2 Weighted sum method

In (Fig. 13) it is possible to see the Pareto front computed with the weighted sum method, both in the design variable space and in the objective function space, compared with the results obtained with the uniform grid.

The weighted sum method is fast, the algorithm is easy to implement and it will give a Pareto front whose density will depend on λ increments, but it is not able to find the entire Pareto front of a non-convex system. It can be efficiently used for problems that are known to be convex.

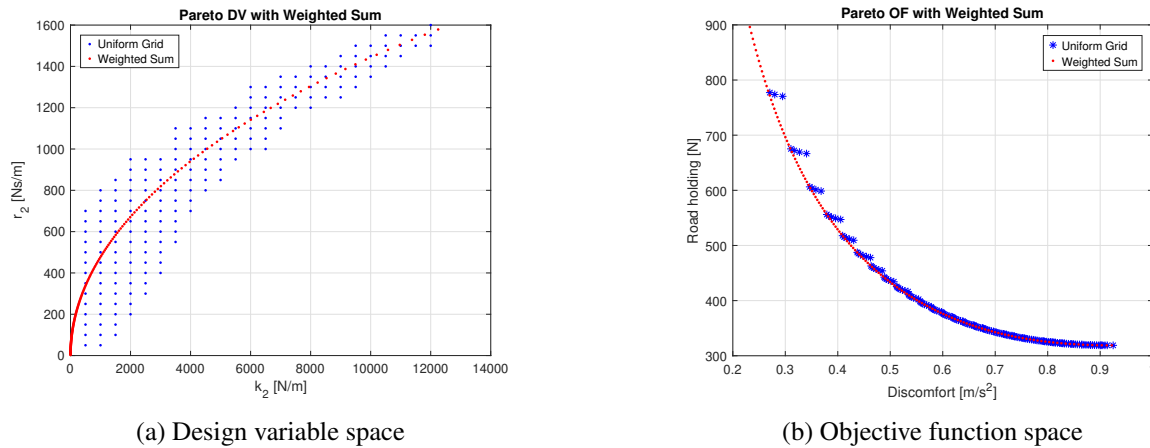


Figure 13: Pareto front computed using weighted sum method

3.2.3 ϵ -constrain method

ϵ -constrain method will help in finding the Pareto front of a non-convex problem (which is not this case) and, depending on the numerical method used, it can give slightly different results and compiling time, moreover it shows a dependence on the initial conditions given both to the design variables and to ϵ . In particular, correct values of ϵ must be chosen to activate a constrain, if the values are wrong and the constrain is not activated, it is possible to only find one single Pareto optimal point, which introduces the need to have at least a rough idea of the OF space shape. Moreover, as for the weighted sum method, the Pareto front density depends on the increments imposed to ϵ when running the algorithm.

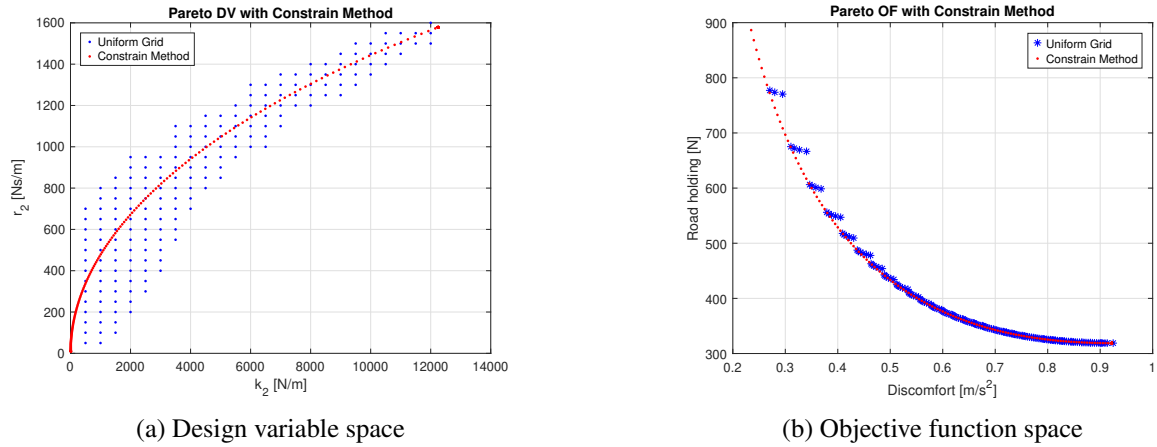


Figure 14: Pareto front computed using constrain method with interior point algorithm

We observed a strong influence of the initial guesses values on the results obtained with the SQP method. After having found a guess which gave suitable results, we noticed some isolated points which appear to be outside the expected Pareto set, probably due to some numerical issues arising at particularly low k_2 values, considering that this phenomenon does not seem to arise when opting for the interior-point method, as visible in (Fig. 15). Apart from that, the results using both numerical methods are exactly the same, perfectly overlapping one another.

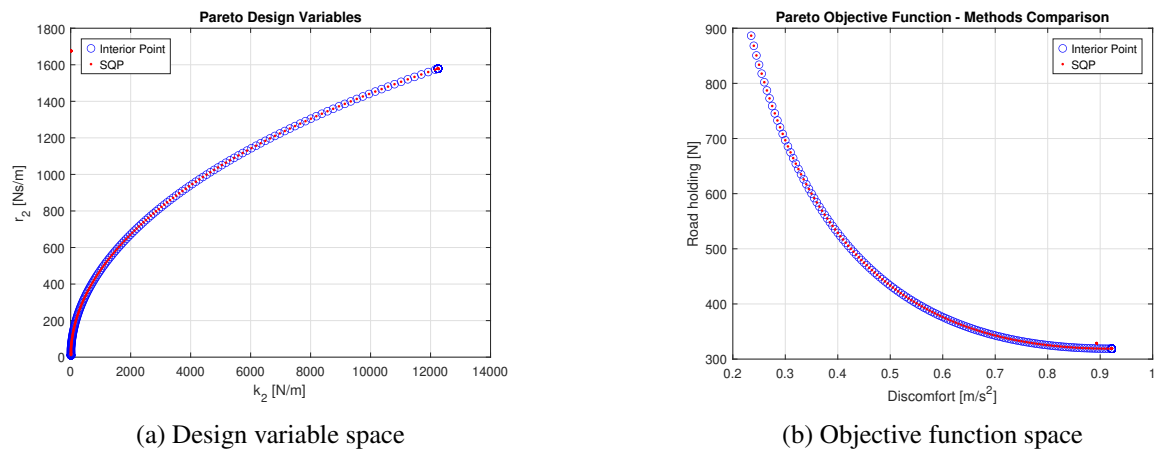


Figure 15: Comparison between sqp and interior point algorithm

3.2.4 Analytical formulation

Using Fritz John optimality condition it is possible to derive an analytical formulation, which is able to describe with a continuous function the Pareto optimal front. This method is valid only if the objective functions are differentiable and usually can be slower and less efficient, since it needs the use of symbolic functions and variables, but it is extremely precise being analytic.

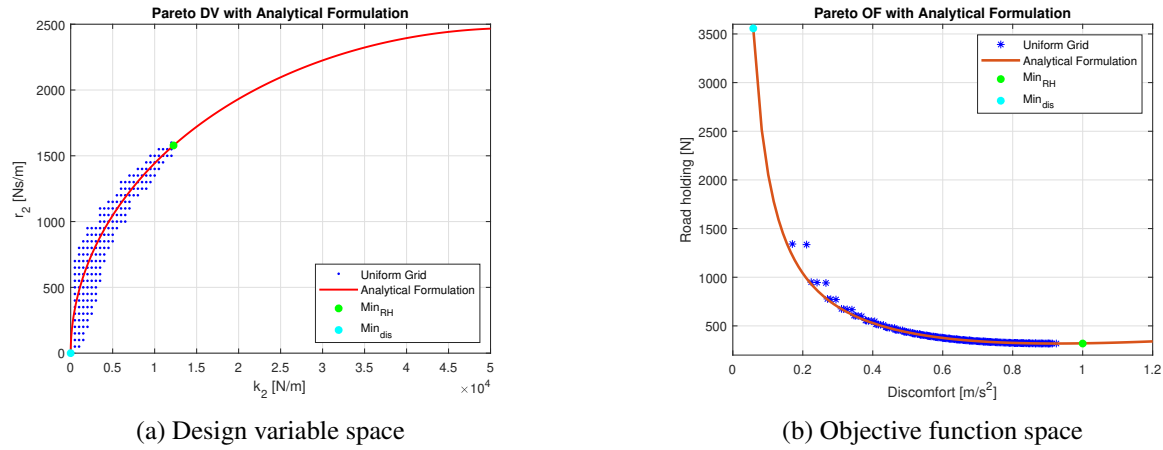


Figure 16: Pareto front computed using the analytical formulation

The continuous function describing the Pareto front has to be carefully used as it can identify as Pareto points also points outside the range given by the smallest values of the objective functions, thus it must be considered only between the minima of each objective function. The minima were found by imposing the gradient of both OFs equal to zero, as shown in (Sec. 3.1.3). Having the discomfort an asymptote in 0, and the minima being equal to $k_2 = 0$, $r_2 = 0$, it was not possible to evaluate numerically the Pareto front function in that point, since it would diverge to infinity, so in (Fig. 17b) the point shown as Min_{dis} , is the smallest value of discomfort that could be computed by the algorithm. From (Fig. 16) it is clear that in the DV space the minimum coming from the analytical formulation are close to the uniform grid ones, though more precise, while in the OF space is clear how much of the real Pareto front is actually missed using the uniform grid at low discomfort, showing how much more precise and broad the analytical approach is.

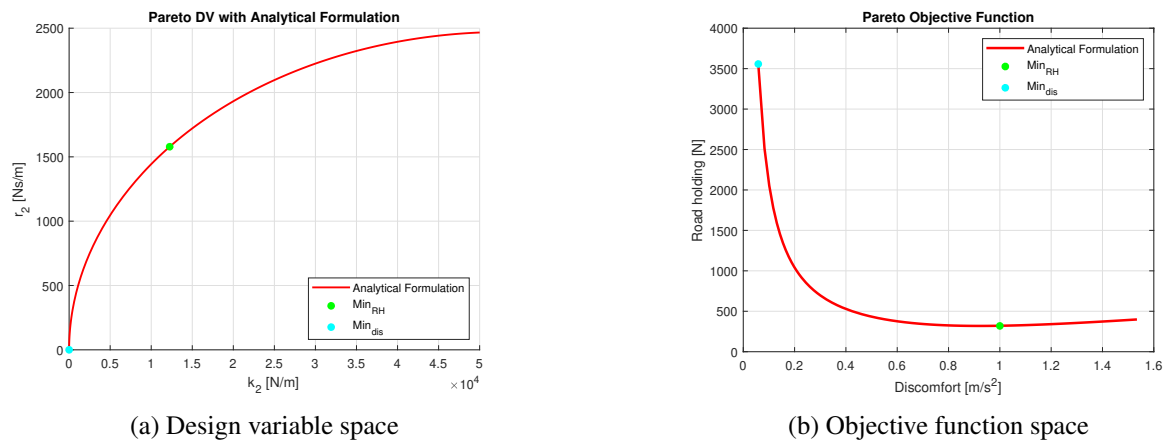


Figure 17: Analytical Pareto front with minimum points

3.2.5 General remarks

In general the same conclusion can be drawn for each different method. The Pareto optimal set, shown in the design variable space, will result more precise and condensed respect to the uniform grid. A single curve can be observed in each case giving a clearer interpretation of the problem, the curve lays in the middle of the grid, which is what can be expected, since the uniform grid would give a region where the solution can be found, while the other methods will give a more precise approximation of the solution (i.e. a line), or even the analytical solution as clearly visible in (Fig. 18a).

The same can be said in the OF space: here the multiple Pareto points identified by the uniform grid method will condense into one single line, this is valid for each other method used. As visible in (Fig. 18b), all the methods will give the same results, being the Pareto front exactly the same in every case. This is probably given by the simplicity of the problem, characterized by a reduced number of design variables and objective functions. The convexity of the problem is also the reason why all the methods seem to coincide, since simpler methods like the weighed sum will not suffer from that.

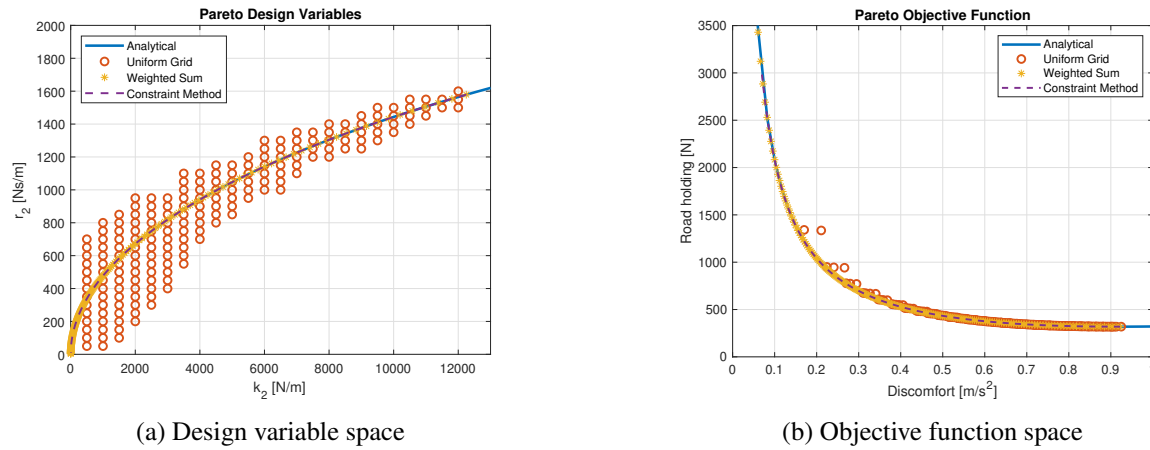


Figure 18: Comparison between all the methods

The only notable difference between all the different methods is how close the Pareto front can get to the actual analytical minima of the objective functions. In (Fig. 18b) it is possible to see that the weighted sum method tends to be the only one reaching toward the analytical minima of discomfort, while the ϵ -constrain method stops a bit before, and the uniform grid is far off; this difference is probably related to how each method tackles the presence of the asymptote and the lower bound of the design variables, especially depending on the discretization of the variables (DVs, λ , ϵ). Regarding the minima of road holding, none of the numerical methods are able to reach it, and they all stop around the same value of 0.93 m/s^2 discomfort.

4 Lab.4 Pareto-optimal set active suspension

4.1 Problem definition

In this analysis the same problem of Lab.2 and Lab.3 was analysed, but, instead of a passive suspension as described in (Sec. 2.1), an active suspension was implemented. The suspension has a sky-hook configuration (Fig. 19), in which an additional active damper is included in the suspension.

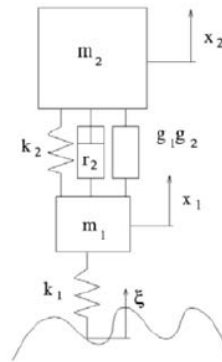


Figure 19: Quarter car model with active suspension

In this configuration the actuator's force is function of the relative displacement between the wheel and the car body and of the car body absolute vertical velocity (Eq. 4.1), where g_1 is an equivalent additional stiffness and g_2 is the damping coefficient of the active damper, which is directly controllable by a control logic. In order to simplify the problem the stiffness and damping of the passive suspension are neglected, assuming $k_2 = r_2 = 0$.

$$F_{actuator} = g_1(x_2 - x_1) + g_2\dot{x}_1 \quad (4.1)$$

The data used in the computations are the same used in (Tab. 4) except for the design variable table, which obviously is different and is reported in (Tab. 5), since now the design variables will be g_1 and g_2 .

Design Variables	Field of variation
g_1	$0 : 400000 \text{ N/m}$
g_2	$0 : 100000 \text{ Ns/m}$

Table 5: Suspension design variables

4.2 Optimization problem

The reasoning behind this optimization problem is the same as the one discussed in (Sec. 2.2), but the formulations of road holding and discomfort are reformulated in (Eq. 4.2) and (Eq. 4.3) to study the specific case of the active suspension.

$$\sigma_{\ddot{x}_2} = A \cdot \sqrt{\frac{k_1 g_1}{m_2 g_2}} \quad (4.2)$$

$$\sigma_{F_z} = A \cdot \sqrt{\frac{g_1 k_1 (m_1 + m_2)^2}{g_2 m_2} - \frac{k_1^2 (2m_1 + m_2)}{g_2} + \frac{k_1^3 m_2}{g_1 g_2} + \frac{g_2 m_1 k_1^2}{g_1 m_2}} \quad (4.3)$$

To efficiently tackle this problem, at first an analysis using the uniform grid method was run to get some first tentative results, obtaining a coarse Pareto front. Then, starting from these results, the ε -constrain method was applied, using discomfort as constrain, with the interior point algorithm, to obtain a clearer, denser and more precise Pareto front.

4.2.1 Uniform grid method

As visible in (Fig. 20) a Pareto front in the OF space, similar to the one of (Sec. 2.3.2), has been obtained. By observing the DV space instead, it seems that the Pareto solutions follow two orthogonal lines, one about horizontal and one about vertical, as better visible in the following section.

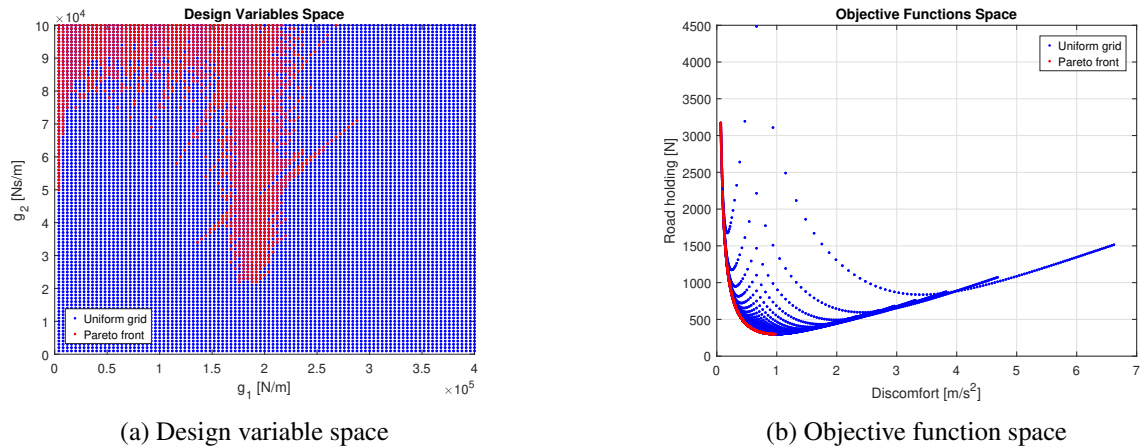


Figure 20: Uniform grid method

4.2.2 ε -constrain method

By observing (Fig. 21), through the ε -constrain method, we got a "clearer version" of the Pareto solutions in the DV space, with a correspondent Pareto front which is coherent and denser with respect to the one obtained through the uniform grid. The peculiar DVs behaviour represented in (Fig. 21a), is related to the OF formulation. Being discomfort inversely dependent on g_2 , it will reach its minima when g_2 is maximum, resulting in the horizontal line in the DV space. Meanwhile the vertical line is given by the formulation of road holding: here g_1 is both at numerator and denominator with grade 1, so there exists an optimal value of g_1 for which RH will be minimum.

The fact that the uniform grid Pareto front in (Fig. 21b) interrupts at lower values of RH, respect to the constrain method, depends on the chosen intervals of the uniform grid DVs.

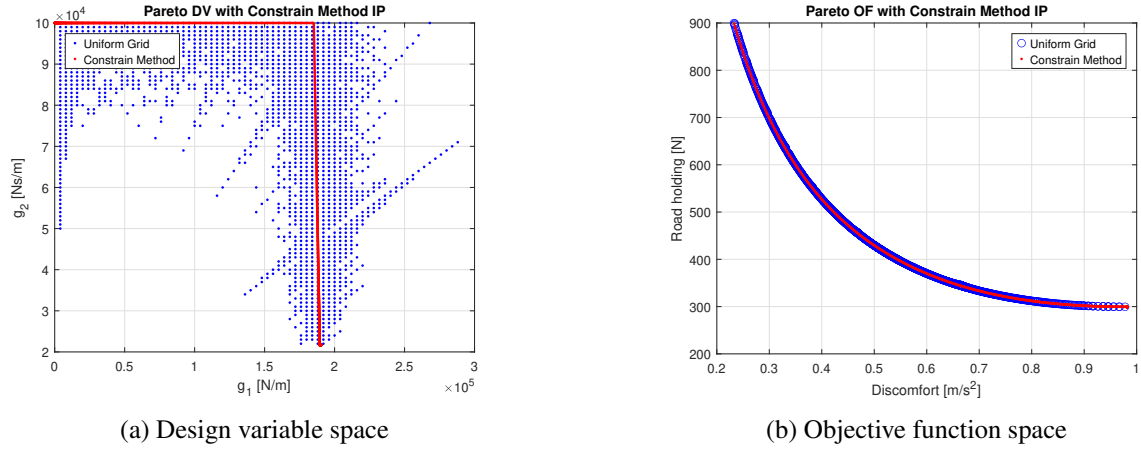


Figure 21: ϵ -constrain method with interior point

4.3 Passive/Active suspensions comparison

If we compare the passive suspension system results with the active ones, as shown in (Fig. 22), we can notice that the two Pareto front are almost perfectly coincident.

This is explainable with the fact that, for the same road conditions, the two different systems will tend to the same optimized values of the OFs, even if the problem formulation is different due to different DV used, leading this to different trends in the DV space. When introducing the active suspension we are "deactivating" the passive one and, since the goal of both suspensions is the same, the two systems should tend to the same optimization when considered separately.

The advantage that the active suspensions introduce is the capability that they have to reach an optimal solution for different working conditions (loads, road roughness, etc.) during the operation, unlike the passive system which, once optimized for a particular condition, if the condition changes, it will not be able to adapt with the same ease reaching another optimal solution.

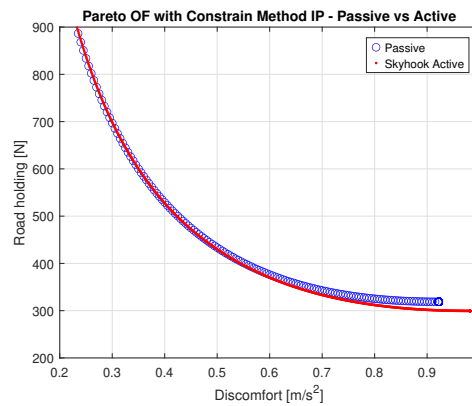


Figure 22: Passive vs Active

5 Lab.5 Genetic Algorithm

5.1 Problem definition

The main idea of the GA steps, takes inspiration from the Darwin's theory of evolution. In particular, in the most complete form, it is composed by the following steps:

1. An initial population of design variables is chosen within a proper domain (e.g. through a uniform grid, random values, etc.)
2. Crossover: the initial population is considered as a set of parents which combine their genes (represented by bits) to generate some children with the intent of increasing fitness (the higher the fitness the higher the probability of being paired). From a population of N parents, a new one of N children is generated
3. Mutation: the genes of a certain specified percentage of children are exchanged to generate some random variability in the DVs
4. Selection: considering parents and children together, the N best (i.e. highest fitness value) DVs are extracted and adopted as the new parents set to be combined
5. The process is iterated from step 2 to step 4 until the proper final set is obtained (according to the specific problem)

Note that in order to properly perform the genes combination and mutation the values defined on a decimal scale need to be converted into a binary scale, in which each bit (i.e. 0 or 1 value) corresponds to a gene.

Also note that the *mutation* step is not necessarily introduced in a GA, but it can be useful to randomly find some new feasible solutions, this helps to avoid having all the solutions clustered in a single small region, spreading them on the entire domain instead, also avoiding premature convergence. Similarly, it is not strictly necessary to perform the *selection* step: it is possible to just use the children set as the new parents variables; the *selection* step makes the problem to converge more quickly, by excluding those children which actually make the solution worse.

A simple optimization problem is faced in this laboratory, that is the maximization of the function:

$$y = 100 - (x_1^2 + x_2^2)$$

The fitness to be maximized, in this case, is defined as the y value itself. While x_1 and x_2 are the design variables.

The other useful data are: a starting population of $N = 100$ individuals, randomly extracted within an interval defined as $[0 \ 5]$ (for both the DVs) and binary values defined through 16 *bits*.

Clearly, the analytical solution is trivial: $x_1 = x_2 = 0$, corresponding to $fitness = y = 100$.

In the following sections, this simple problem is solved by gradually introducing the aforementioned steps. The stopping condition for the cycle is in this case a maximum number of 200 *iterations*.

5.2 Results

5.2.1 No mutation, no selection

The first case, introduces steps 1 and 2 only, with a 100 % probability of crossover, meaning that all the selected crossovers are effectively performed.

The results are shown in (Fig. 23).

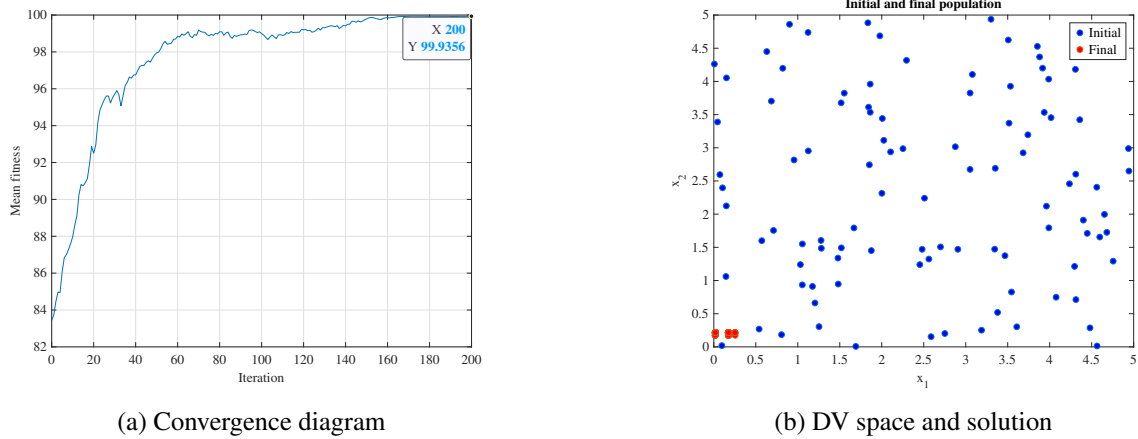


Figure 23: No mutation, no selection

It can be observed that a good but not extremely precise result is obtained. Moreover, the convergence is pretty slow, considering that an acceptable mean fitness is reached only after about 160 iterations.

5.2.2 Mutation, no selection

The second case introduces step 3, through a 2 % probability of mutation, which causes some genes to randomly change their value.

The results are shown in (Fig. 24).

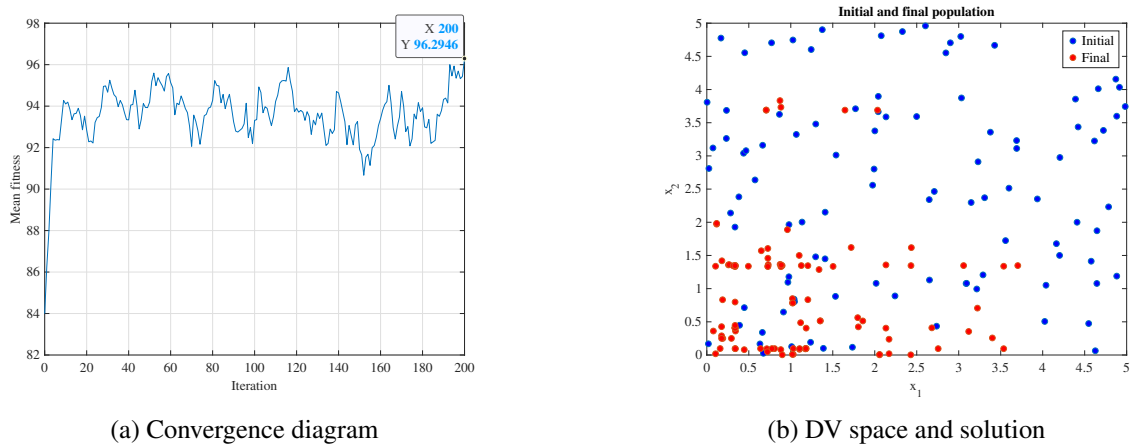


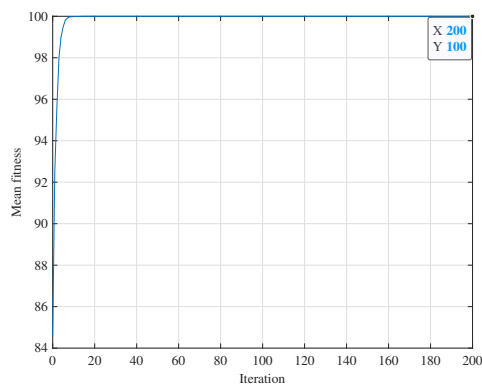
Figure 24: Mutation, no selection

In this case, the effect of random mutation without a proper selection is quite evident, with a final mean fitness which is improved with respect iteration 0 but with a definitely not precise result (actually worse than the previous one).

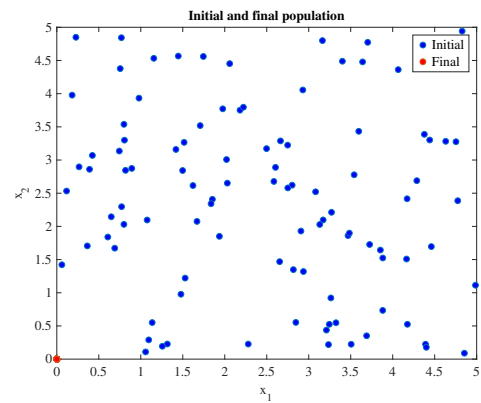
5.2.3 Mutation, selection

Finally, step 4 is introduced too. In this last case, the best individuals within the total parents-children set are selected iteration by iteration.

The results are shown in (Fig. 25).



(a) Convergence diagram



(b) DV space and solution

Figure 25: Mutation, no selection

A huge improvement is observed in this latter case, with a monotonically increasing mean fitness, a convergence reached after about 10 iterations only and an extremely precise result (actually, the mean fitness is exactly $y = 100$ and the solution is a 100 individuals set composed by all 0 values).

6 Lab.6 Double wishbone suspension design with GA

6.1 Problem definition

The optimization problem of a double wishbone suspension (Fig. 26) kinematics was tackled, using a Genetic Algorithm (i.e. GA). The objective of the minimization problem was to reduce the distance of two kinematic quantities with respect to their reference values. In particular, the camber gradient and the roll centre height were studied: both quantities were influenced by the angles θ_1 and θ_2 , which represent the inclinations with respect to the horizontal axis of the lower and upper arm respectively.

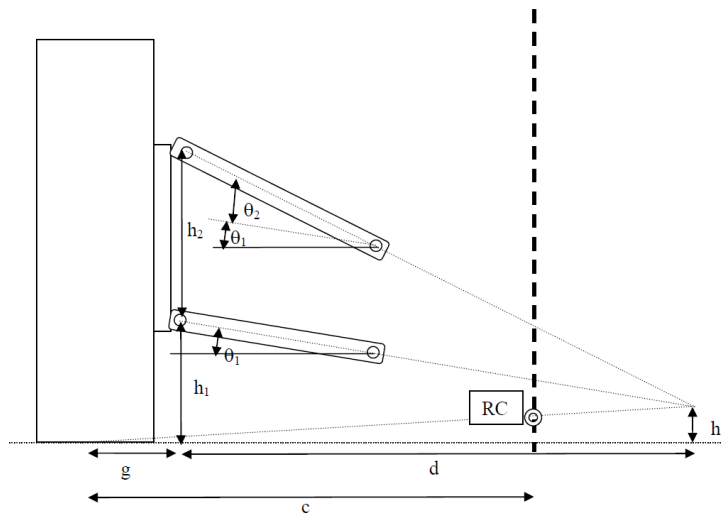


Figure 26: Suspension scheme

The camber gradient (Eq. 6.1) depends on the distance between the centre of the wheel from the arms connections g and the distance from the arms connection to the virtual hinge located inside the vehicle body d , evaluated at a certain wheel vertical displacement.

$$\Delta\gamma = \frac{\Delta z}{d + g} \quad (6.1)$$

The roll centre height (Eq. 6.2) depends again on g and d , but also on the virtual hinge height h and the horizontal distance from the wheel centre to the roll centre c .

$$RCH = \frac{h}{d + g} c \quad (6.2)$$

Where:

$$\begin{cases} d = \frac{h_2}{\tan(\theta_1 + \theta_2) - \tan(\theta_1)} \\ h = h_1 - d \cdot \tan(\theta_1) \end{cases} \quad (6.3)$$

The given data are collected in (Tab. 6)

h_1	0.19 m
h_2	0.27 m
g	0.05 m
c	0.87 m

Table 6: Suspension dimensions

6.2 Optimization problem

The chosen design variable were the angles θ_1 and θ_2 , whose field of variation is reported in (Tab. 7). While the kinematic targets to be aimed at are collected in (Tab. 8), where the camber gradient is being evaluated at a wheel travel of $\Delta\bar{z} = 50\text{ mm}$.

DV	Field
θ_1	$-9^\circ : -1^\circ$
θ_2	$+5^\circ : +15^\circ$

Table 7: Design variables

Quantity	Target
RCH_{target}	100 mm
$\Delta\gamma_{target} \Delta\bar{z}$	$+2^\circ$

Table 8: Objective functions targets

In order to reach the targets set in (Tab. 8) the chosen objective functions were based on a least squared error minimization approach, where the aim was to minimize the error between the computed value and the reference one. The formula used are shown in (Eq. 6.4).

$$\begin{cases} f_1 = \frac{(RCH - RCH_{target})^2}{(RCH_{target})^2} \\ f_2 = \frac{(\Delta\gamma|\Delta\bar{z} - \Delta\gamma_{target}|\Delta\bar{z})^2}{(\Delta\gamma_{target}|\Delta\bar{z})^2} \end{cases} \quad (6.4)$$

6.3 Genetic Algorithm

Genetic algorithms work by evaluating different combinations of the starting DV, selecting only the best solutions, using probabilistic transition rules to combine the DVs. It is an iterative algorithm were, in each cycle, several passages are carried out, both using a decimal definition of the DVs, or a binary one.

The algorithm starts by creating randomly an initial population of DVs which are then evaluated using the OFs to extract a fitness of each variable. Then a `while` loop runs until all the DVs have a fitness equal to 1, meaning that a single Pareto front has been found.

6.3.1 Evaluation

Particular attention must be given to the evaluation method: after evaluating all the DVs via the OFs (Eq. 6.4), an internal cycle was run to extrapolate the value of fitness of each DV: at each internal iteration a Pareto front was computed, which defined the rank of the DVs that composed it, then the DVs corresponding to that specific Pareto set were removed and the process was then reiterated. In particular, the first Pareto set corresponded to rank 1, after having removed it from the solutions, the second Pareto set corresponded to rank 2, and so on, until all the DVs were described through a specific rank value. The fitness of each DV was then defined as the reciprocal of the DV rank, meaning that multiple DVs will have the same fitness.

6.3.2 Cycle

In each cycle these computations are carried out, where the parents are the variable at the previous iteration, while the sons at the current iteration:

1. Coding: the DVs are coded into a binary coding using a certain number of bits.
2. Reproduction: the DVs are paired two by two in a probabilistic way based on the fitness of each DV (the higher the fitness the higher the probability of being paired).
3. Crossover: the selected pair of parent DVs, in binary coding, have a part of their string cut and exchanged between the couple, creating sons.
4. Mutation: to avoid that all the DVs end up clustered in a single small region the strings of the sons are randomly muted by altering some bits. This mutation is controlled by a probability of mutation parameter to decide whether to mutate or not.
5. Decoding: the new generated DVs, together with the old ones (i.e. sons and parents) are then decoded back from binary to decimal.
6. Evaluation: both parents and sons are evaluated together (Sec. 6.3.1) and their fitness is ranked from the fittest to less fit.
7. Discard: only the most fit DVs are selected, finally obtaining a number of variables equal to the starting one. This is done to avoid the creation of a too wide population, which can hinder both the results and computational time, also making in this way the convergence to be quicker, by selecting only the best individuals.
8. Updating: the DVs that survive the discarding will become the new starting DVs which enter the coding part (step 1).

6.4 Results

In the following, different results, corresponding to different parameters, are reported.

In all the cases, both θ_1 and θ_2 are represented by 16 samples, resulting in a population of $16 \cdot 16 = 256$ initial individuals.

Note that, to exclude from the comparison any other external influence, the same (randomly generated and then combined together) initial population has been considered. This population is plotted in (Fig. 27)

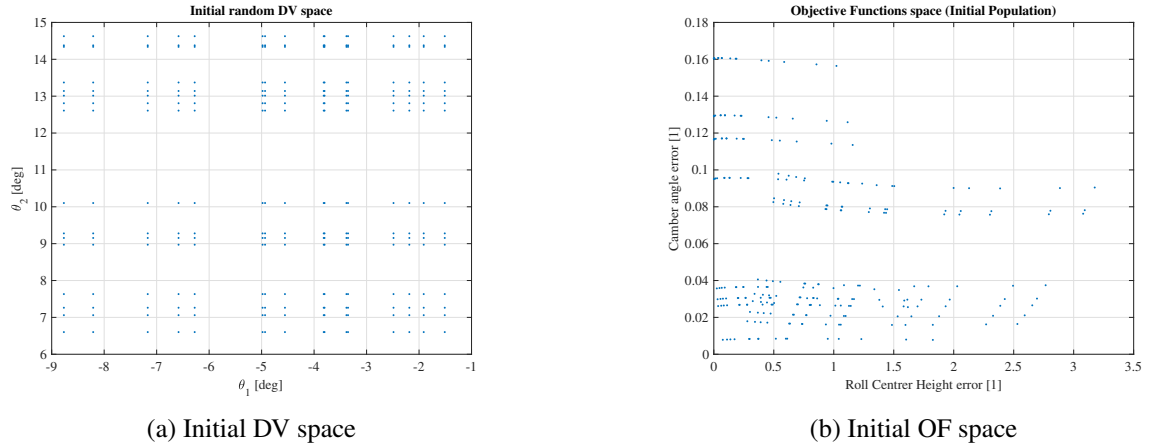


Figure 27: Initial random population

Because of the probabilistic nature of the process, all the cases have been run many times with the same parameters, to ensure the robustness of the presented results.

6.4.1 Number of bits: 10 | Mutation probability: 2.23 %

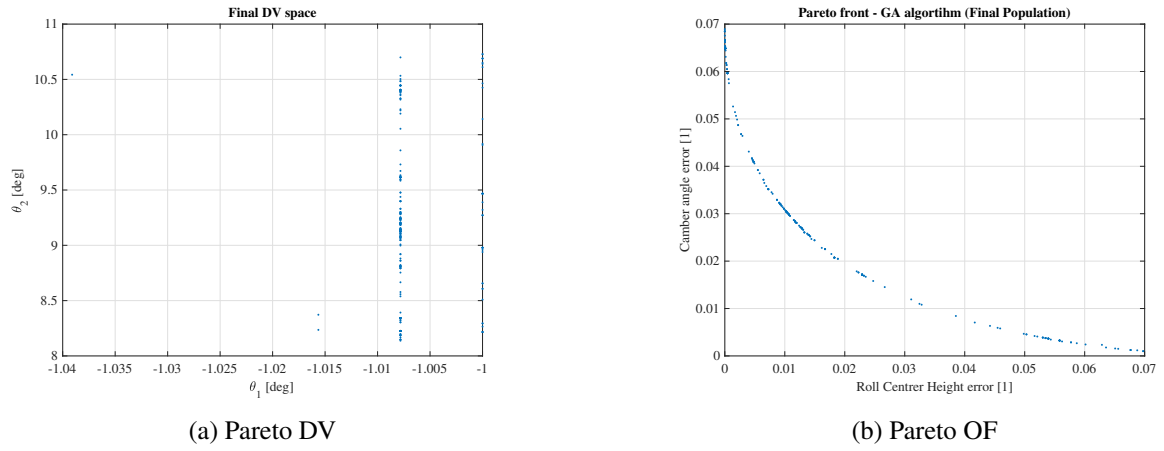


Figure 28: Pareto set

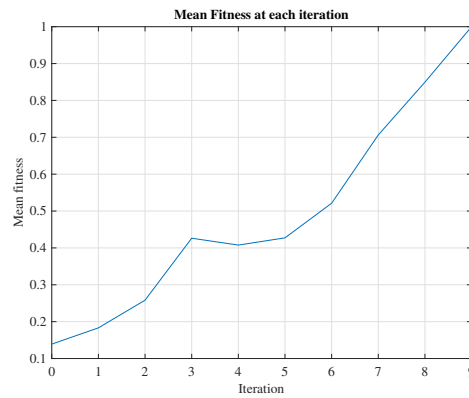


Figure 29: Convergence plot

Convergence is reached in 9 iterations, with a typical shape of a 2-OFs Pareto set, and the DVs concentrated around a value of $\theta_1 = -1 \text{ deg}$ and θ_2 spanning from 8 to 10.75 *deg*.

This result is more or less valid for all the following cases and it has been checked and confirmed also through a "classical" uniform grid (without GA) approach, as can be also seen by looking at (Fig. 27), where the shape of the Pareto front and the values are coherent with the ones obtained here.

6.4.2 Number of bits: 5 | Mutation probability: 2.12 %

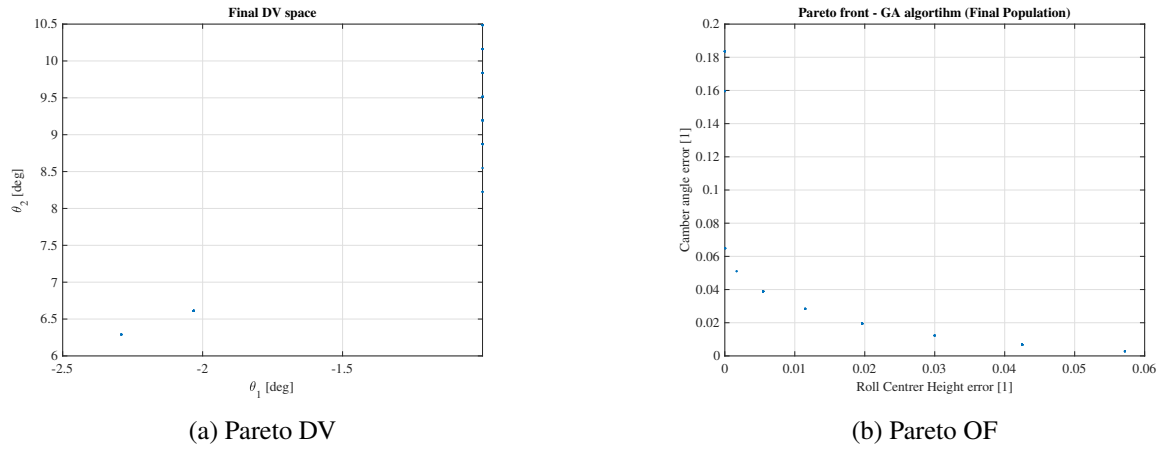


Figure 30: Pareto set

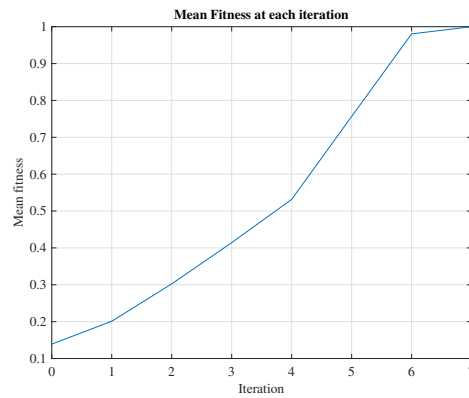


Figure 31: Convergence plot

A faster but coarser convergence is reached in this case. In particular, a lower number of iterations is needed (7 instead of 9), but the reduced number of bits reduces also the discretization density of the DV space. Indeed, within the same interval, with 10 bits there's the capability of reproducing $2^{10} = 1024$ different values, while $2^5 = 32$ values can be represented with 5 bits only. Notice that to represent 16 values of a DV at least 4 bits are needed.

This lower discretization capability is well visible both in the DV space and in the OF space.

6.4.3 Number of bits: 20 | Mutation probability: 2.18 %

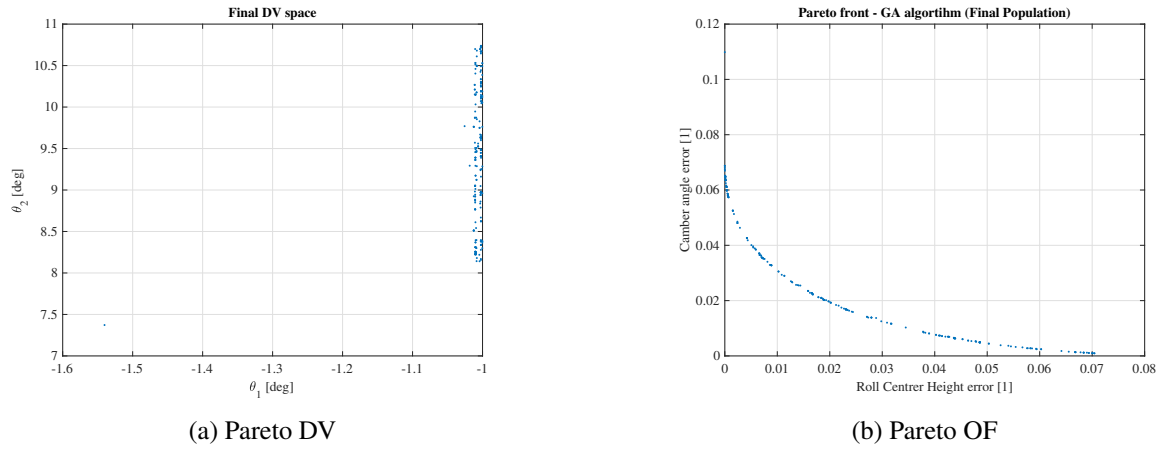


Figure 32: Pareto set

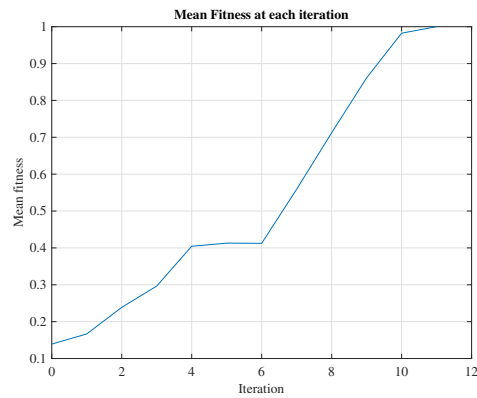


Figure 33: Convergence plot

Not substantial differences are highlighted here with respect to (Sec. 6.4.1), apart from a slightly slower convergence (11 iterations instead of 9) and a "less discretized" samples set. Also, less concentrated values around a specific θ_1 are observed.

A further increasing in the number of bits seems to produce no better results, as also the Pareto front seems to have a similar distribution of values.

6.4.4 Number of bits: 5 | Mutation probability: 10.00 %

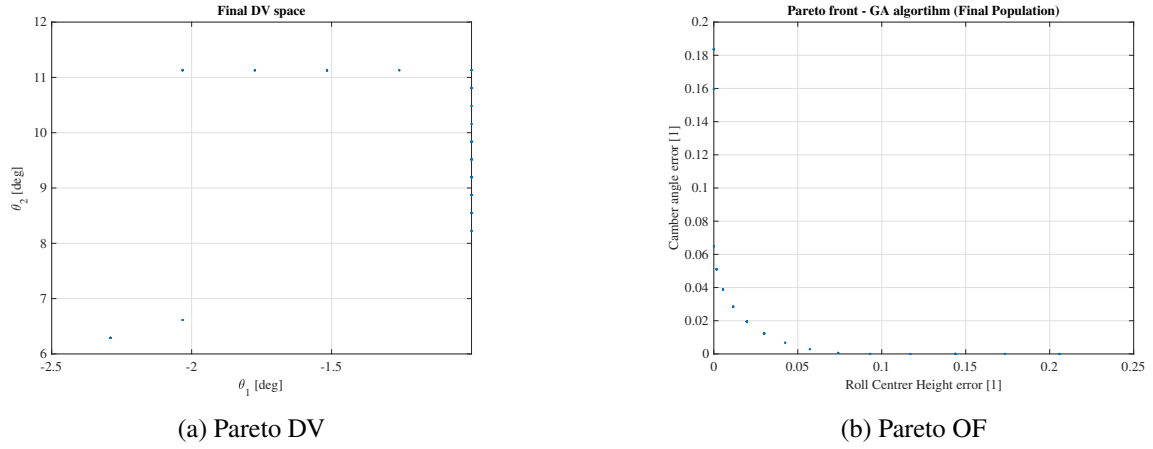


Figure 34: Pareto set

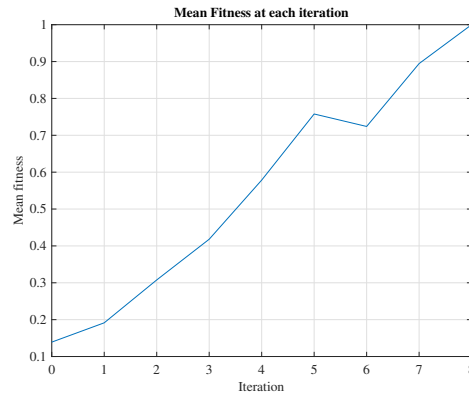


Figure 35: Convergence plot

Due to the frequent generation of random mutations, more individuals move far from the solutions of the Pareto set of the reference case.

At the same time, even if the Pareto set in the OF space is similar to the one of (Sec. 6.4.1), the DVs are less concentrated around $\theta_1 = -1 \text{ deg}$, being this representative of the increased probability of random generation of totally new individuals. This effect is also visible in the extended boundaries of the Pareto set in the OFs space.

6.4.5 Number of bits: 10 | Mutation probability: 0.16 %

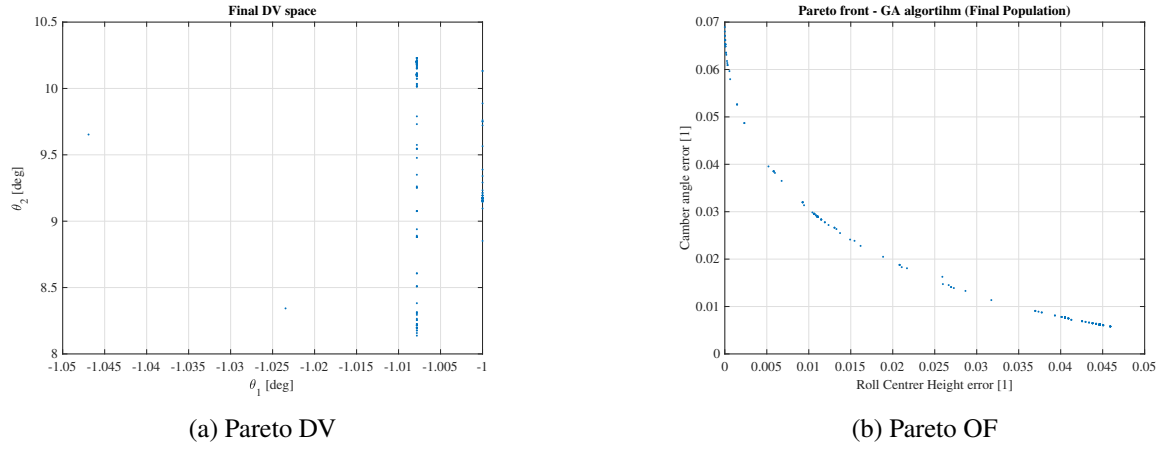


Figure 36: Pareto set

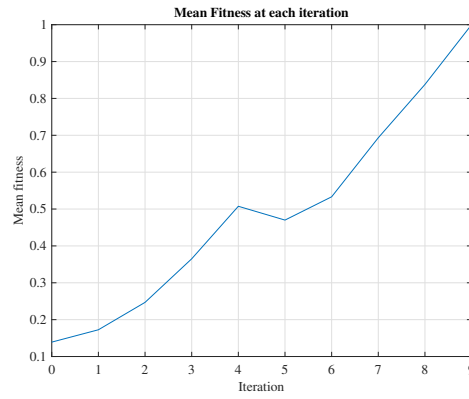


Figure 37: Convergence plot

In this case, with a convergence rate similar to the one of (Fig. 29), the differences lay in the more concentrated individuals, which tend to congregate in blocks in the OFs space, and in the fact that all the designs are precisely positioned at the same value of θ_1 , being these two effects representative of the lower probability of generating totally different individuals, far from the through the mutation process. Meaning that probably there are around two block of variables with the same schema, resulting in very similar values of θ_1 .

6.4.6 Number of bits: 16 | Mutation probability: 1.58 % 50x50 DV linspace

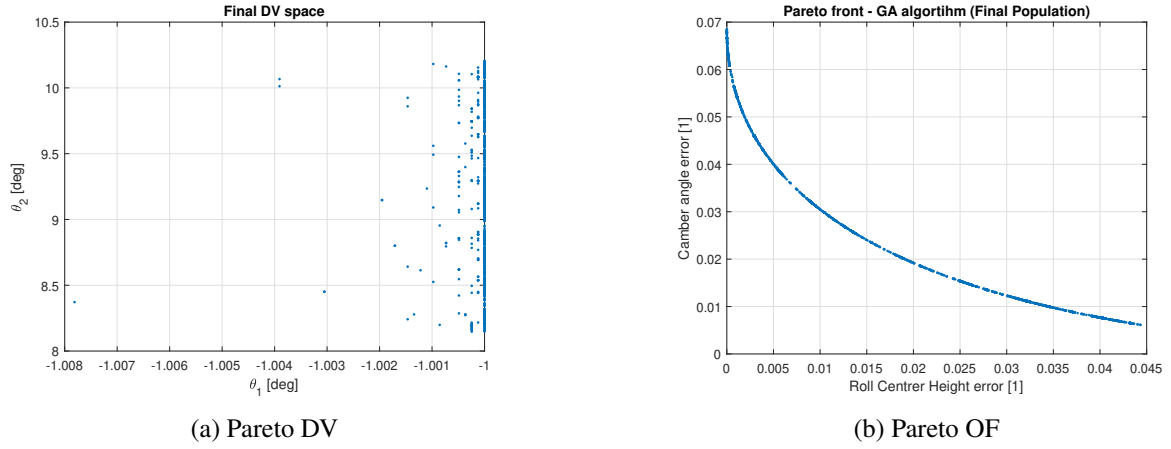


Figure 38: Pareto set

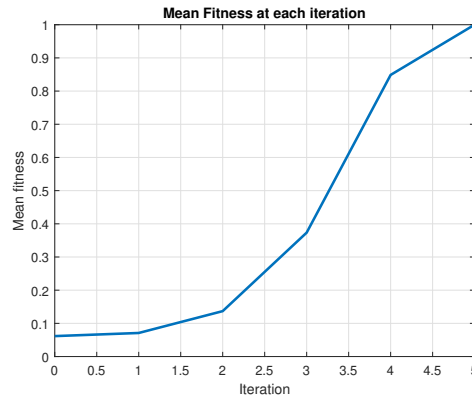


Figure 39: Convergence plot

A new simulation with a different approach was run with the aim of finding a finer Pareto front. The dimension of the design variables was increased to 50 and it was not any more created randomly but through a `linspace` command in Matlab.

Different runs were carried out to investigate the algorithm stability and it was found to converge very fast: 5 iteration each run. Some times some θ_1 variables appeared with a "high" value of around -1.7, probably due to mutation, alongside with some very low RCH values.

In general, the solution, respect to the other cases presented before, presents a way finer Pareto front due to the increased number of individuals and also a faster convergence. The DV space is more compact with ranges of both θ_1 and θ_2 decreasing: in particular θ_1 now spans from -1° to just -1.0078° , with most of the values concentrated around -1° ; θ_2 now spans from 8° to 10.25° .

The main reason why the convergence is reached faster seems to be the initial creation of the DVs, where a uniformly spaced grid seems to ease the algorithm.

6.4.7 Number of bits: 16 | Mutation probability: 1.58 % 50x50 DV rand

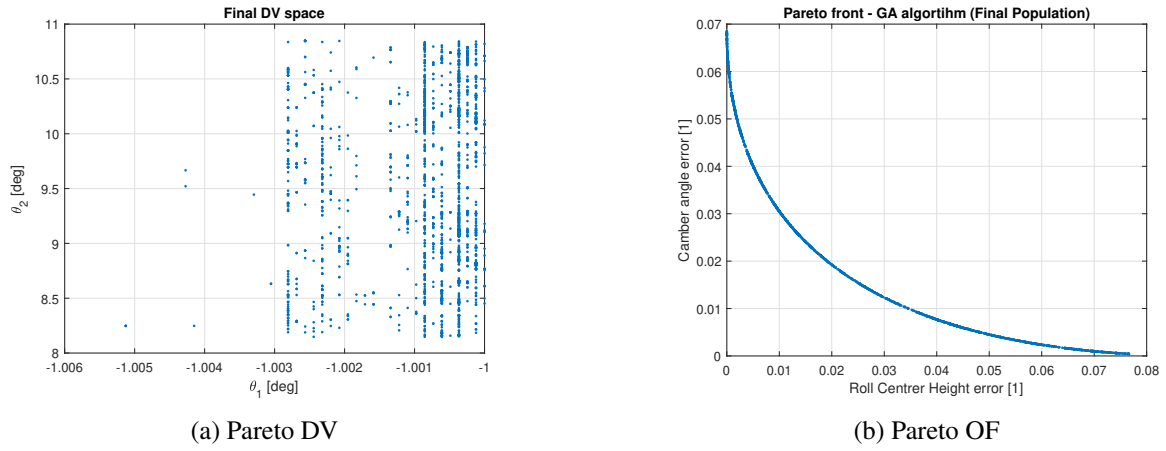


Figure 40: Pareto set

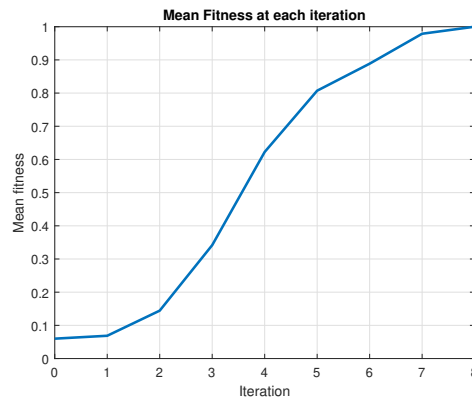


Figure 41: Convergence plot

Basically the same conclusion can be drawn here (where a rand command was used to create the DV) as in (Sec. 6.4.5): the solutions have basically the same ranges: slightly smaller range of θ_1 , slightly wider range of θ_2 . A wider distribution in the DV space appears, allowing for more diversification in the variables domain; instead the OF space is even finer.

The convergence is slower than in (Sec. 6.4.5) but still in line with all other experiments, converging generally with 8 iterations, even if with a way higher number of individuals.

7 Lab.7 Neural Network creation and training

7.1 Problem definition

Artificial Neural Networks are part of a wider family of techniques, namely the Global Approximation techniques. This techniques aim to substitute physical models with approximated purely mathematical ones.

These Artificial Neural Network resemble how the human brain works: they consist of a set of units (called neurons), connected by weighted lines (axons), in which each neuron to provide a result has to overcome an activation threshold and transofms a series of inputs in a series of outputs thanks to a non-linear activation function.

In these laboratories, the chosen Network type is the so called Multi Layer Feedforward Network. It is characterised by an input, an output and a number of hidden layers (one or two in our case); in each of these layers, neurons are present and connected only to neurons of the next layer, such that communications can only move towards the output, without any feedback. Each neuron receives weighted inputs from neurons at the previous layer and is characterised by a non-linear transformation function.

Training the Network is the most important part of the creation of an ANN. The adopted training method is back propagation of the quadratic error at each previous neuron layer, which is divided in three steps:

1. A given input is sent along the network and its output is computed, this output is compared with a desired or true output, computing the error.
2. The error gradients of the neurons are back propagated starting from the output, computing the weight for each neuron connection
3. Each weight is iteratively updated based on the steepest descent along the gradient of the errors surface.

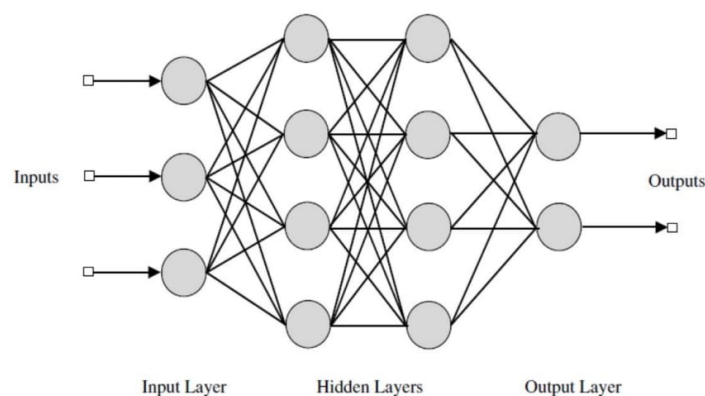


Figure 42: Feedforward ANN example

In this laboratory the effect of different training methods and parameters was investigated, studying the effects and problems of certain solutions.

7.2 Training example results

In this section, the results for two different experiments are presented, showing different behaviours and typical problems of ANN training.

7.2.1 Noisy sine

A sine function with numerical noise is used as output set points to use during the network training, three different situations were analysed. The trained networks were then used with a finer set of inputs.

The network was initialised with one hidden layer with eight neurons inside, using as activation function the tan-sigmoid transfer function and a maximum number of validation fails equal to six.

1. Overtraining: Using the Levenberg-Marquardt training method, which is a deterministic method based on the gradient steepest descent, where the weights are chosen solving a minimisation problem, the network was forced to overtrain, using the whole output as training set and by using a high number of epochs. This resulted in a set of network outputs which well follows the initial training set, minimising the distance between the function and the training points, but obtaining a not general result whose shape is highly different with respect to the original sine function, (Fig. 43a).
2. Validation: Using again the Levenberg Marquardt method with less number of epochs, a part of the training set is used to validate the training while it is taking place. In this case 20 % of the training set is used to correct the neuron's connection weights during training. This results (Fig. 43b) in a more generic network, with an output which is closer to the real function, but still not acceptable. Validation works by giving as input to the just trained NN the inputs of the validation set, to see how much the NN outputs differ from the ones of the validation, computing an error. If this error increases for a certain number of iterations then the training is stopped.
3. Bayesian method: The training method was changed to the Bayesian one, where the training is carried out with a probabilistic approach, estimating the weights of the connections, still using a part of the training set for validation purposes. This method is more suited for noisy signal as (Fig. 43c) shows: the network output is very close to the original function and even if the amplitude of the sine is not respected in some points, the trend is precise. Moreover this method proved to be more robust, as for different random noises the result was always acceptable, while sometimes the LM method could not produce satisfactory results.

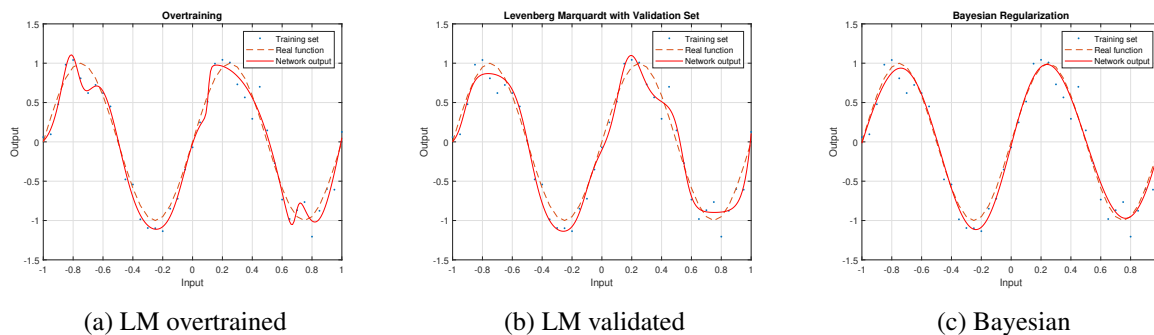


Figure 43: Noisy sine

7.2.2 Exponential function

A second study was made on a different exact function, which received as input a uniform grid. The output of this function was again modified by random noise, to then use it to train the network.

The equation is:

$$z = xe^{-x^2-y^2}$$

The influence of the network architecture on the results was analysed both graphically and by looking at the mean square error between the exact solution and the solution computed via the neural network.

<i>Archit.</i>	MSE_{noise}	MSE_{NN}
[5]	$4.054 \cdot 10^{-4}$	$1.625 \cdot 10^{-3}$
[10]	$3.915 \cdot 10^{-4}$	$1.615 \cdot 10^{-4}$
[20]	$3.872 \cdot 10^{-4}$	$4.571 \cdot 10^{-5}$

<i>Archit.</i>	MSE_{noise}	MSE_{NN}
[5,5]	$3.983 \cdot 10^{-4}$	$1.876 \cdot 10^{-5}$
[10,10]	$3.789 \cdot 10^{-4}$	$3.783 \cdot 10^{-5}$
[3,4]	$4.113 \cdot 10^{-4}$	$7.441 \cdot 10^{-5}$

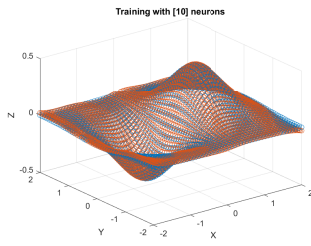
Table 9: MSE using Levenberg-Marquardt

In (Tab. 9) and (Tab. 10) the second column represent the MSE between the exact function and the one with random noise, as it could be expected the error is approximately the same each time. Some oscillations are due to the random nature of the training points creation process. In (Tab. 9) is shown that in the case of only one hidden layer the MSE is highly dependent on the number of neurons, decreasing with increasing number of neurons, this is also shown by the regression factor, starting from $R_5 = 0.9714$ and $R_{10} = 0.9898$ it improves up to $R_{20} = 0.9921$. When two hidden layers are used the MSE is more stable and homogeneous, moreover it was at the same order of magnitude of the best case with one layer. Changing training method to the Bayesian the MSE results in the same order of magnitude (Tab. 10), but the shape of the exact function is followed better, in particular in the peaks, particularly visible in (Fig. 45c) and (Fig. 47c).

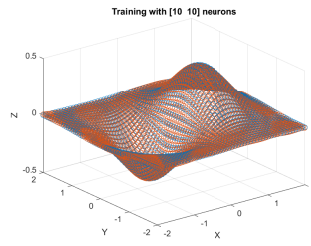
<i>Archit.</i>	MSE_{noise}	MSE_{NN}
[10]b	$4.057 \cdot 10^{-4}$	$6.635 \cdot 10^{-5}$
[10,10]b	$4.018 \cdot 10^{-4}$	$2.380 \cdot 10^{-5}$
[3,4]b	$4.079 \cdot 10^{-4}$	$7.945 \cdot 10^{-5}$

Table 10: MSE using Bayesian

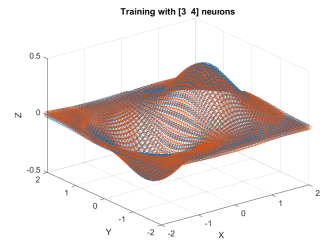
In the following plot the blue points represent the exact function, while the red ones represent the the output of the neural network.



(a) [10] layers

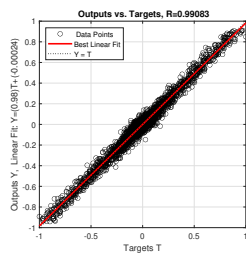


(b) [10,10] layers

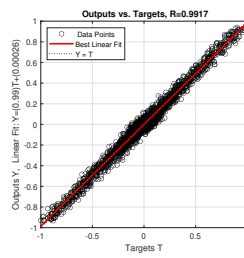


(c) [3,4] layers

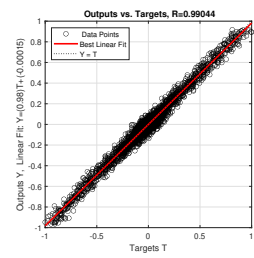
Figure 44: Levenberg-Marquardt method



(a) [10] layers

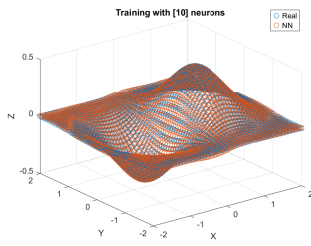


(b) [10,10] layers

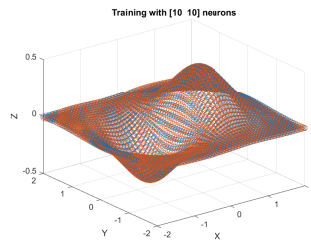


(c) [3,4] layers

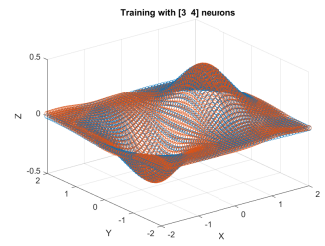
Figure 45: Regression plots using Levenberg-Marquardt method



(a) [10] layers

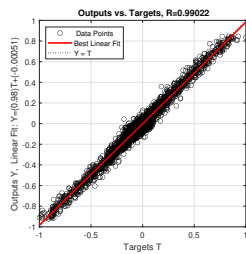


(b) [10,10] layers

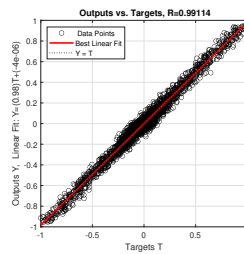


(c) [3,4] layers

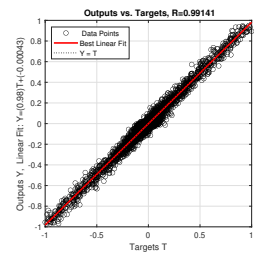
Figure 46: Bayesian method



(a) [10] layers



(b) [10,10] layers



(c) [3,4] layers

Figure 47: Regression plots using Bayesian method

After this analysis it was deducted that, to optimize the results and minimize the errors of the ANN, no clear and unique way of defining the network architecture exists. Only general guidelines can be drawn:

- Usually Bayesian training works better than Levenberg-Marquardt with noisy signals, following better the function's shape. Depending on the case it is able to represent better the peaks or the edges of the function.
- In the case of only one hidden layer, increasing the number of neurons improves the MSE and the regression factor, but the training is longer and more demanding, and the risk of overtraining increases.
- Two hidden layers usually have a more stable MSE and no correlation has been observed between the number of neurons per layer, the best MSE was given by the network with fewer neurons.

8 Lab.8 - Multi-objective handling optimization

8.1 Problem definition

In this laboratory, a three-objective-functions optimization, in two design variables has been faced. In particular, handling was the focus of the optimization process, in which the lateral acceleration, the handling diagram curvature and the yaw rate variation during a throttle release manoeuvre, have been investigated in function of the centre of gravity position and of the roll stiffness.

In particular, the maximum lateral acceleration in the handling diagram was to be maximized (Fig. 48), the maximum curvature of the handling diagram was to be minimized (Fig. 49), the yaw rate variation, one second after the manoeuvre (throttle release while turning at 80 km/h with $a_y = 7 \text{ m/s}^2$) with respect to the initial value, was to be minimized (Fig. 50).

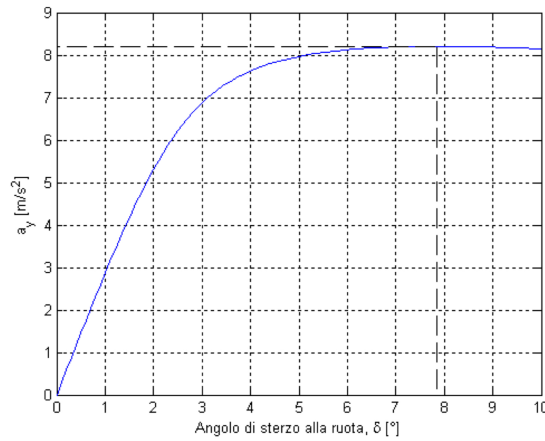


Figure 48: Handling diagram - Maximum lateral acceleration

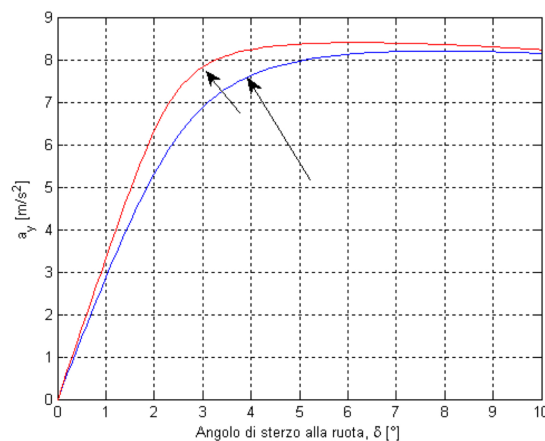


Figure 49: Handling diagram - Minimum curvature

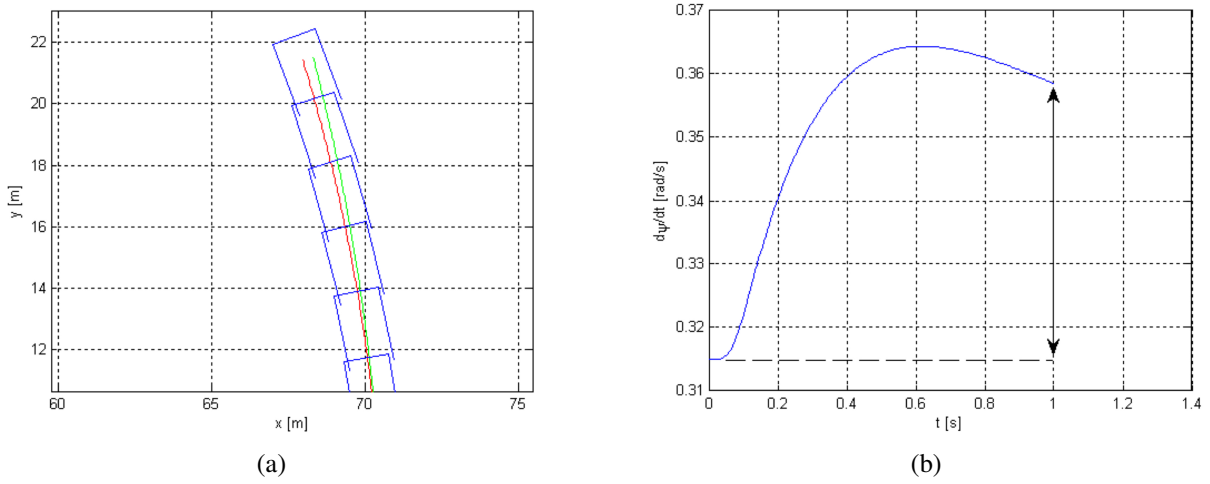


Figure 50: Throttle Release - Minimum yaw rate variation

The assigned values for the design variables are reported in (Tab. 11)

	Min	Max
a [mm]	700	850
H [-]	0.6	1.2

Table 11: Design Variable domains

where a is the center of gravity distance from the front axle and H is the normalized roll stiffness, meaning that $H = 1$ corresponds to no changes with respect to the original value.

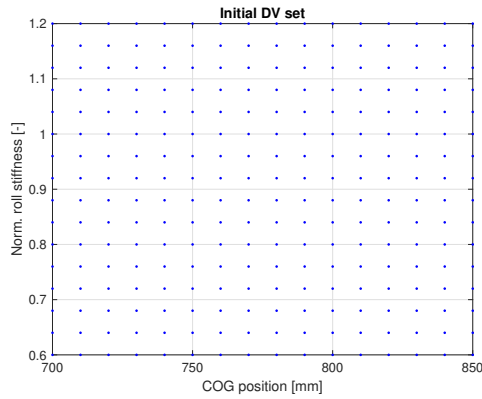
An initial uniform grid was given, with 256 combinations of the design variables and the corresponding objective functions, previously computed through the *VI-CarRealTime* software exploiting a *compact-car* model.

Note that all the given data were to be minimized: yaw rate variation in the first column, $-a_{y,max}$ in the second column, maximum absolute value of the second derivative of the handling diagram (curvature) in the third column.

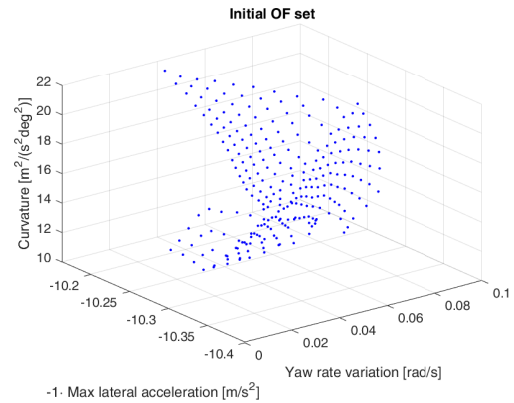
These data, which will be used as training set for the Neural Network, are plotted in (Fig. 51)

In the following sections, a Neural Network will be trained and exploited to create a finer OF space with its 3D Pareto set. The concepts regarding the Neural Networks are available at (Sec. 6.4.7); the concepts regarding the three dimensional Pareto front area available at (Sec. 2.3.3).

All the requests and results of this laboratory are reported in (Sec. 8.2).



(a) Design Variables



(b) Objective Functions

Figure 51: Throttle Release - Minimum yaw rate variation

8.2 Requests and results

8.2.1 Requests 1 and 2 - Spearman Correlation

The Spearman Correlation is a key tool that allows to understand if a function shows a certain level of correlation with another one, such that for instance when the first function decreases, the second one decreases accordingly (direct correlation). If two functions are characterized by such correlation, it is possible to exclude one of the two from the optimization analysis, thanks to the knowledge of the same trend and of the considered objective functions.

Being our problem a three-dimensional one, we can exploit the Spearman coefficients to assess the possibility of treating the problem as a two-objective-functions one. In particular, a Spearman coefficient near to 1 indicates a high level of correlation and justify the removal of one of the correlated OFs.

By means of the MatLab function `corr`, the coefficients are computed and reported in (Tab. 12), for the three given data sets.

corr.	OF1	OF2	OF3
OF1	1	-0.0933	0.6186
OF2	-0.0933	1	0.6217
OF3	0.6186	0.6217	1

Table 12: Spearman Correlation Coefficients

Unfortunately, apart from the diagonal of the matrix which corresponds to the correlation of each function with itself, none of the other coefficients are close to 1: our problem has to be effectively treated as a three-OFs one. No further information on the P-value is required due to uncorrelated dataset.

8.2.2 Requests 3 and 4 - Neural Network training

For the training procedure of the Feed Forward Neural Network, different parameters and solutions have been taken into account, and the resulting sets have been compared. As usual, due to the generation of non-identical results also with the same training parameters and set, the procedure was repeated a few times, to be sure that the reported results were representative of the parameters themselves.

In (Tab. 13), all the parameters combinations are reported. For each combination two different training parameters have been tried: the Levenberg-Marquardt (`trainlm`) and Bayesian Regularization (`trainbr`). In all the cases, 80% of samples were used for training the network and 20% for verifying the generalization properties.

	Hidden neurons Layer 1	Hidden neurons Layer 2	Algorithm
Case 1	5	0	BR
Case 2	5	0	LM
Case 3	10	0	BR
Case 4	10	0	LM
Case 5	20	0	BR
Case 6	20	0	LM
Case 7	3	4	BR
Case 8	3	4	LM
Case 9	5	5	BR
Case 10	5	5	LM
Case 11	10	10	BR
Case 12	10	10	LM
Case 13	12	10	BR
Case 14	12	10	LM
Case 15	15	15	BR
Case 16	15	15	LM
Case 17	20	20	BR
Case 18	20	20	LM

Table 13: Training parameters combinations

In the following, all the results are shown, in terms of training performances, regression coefficients and comparison between training and trained sets.

After the comparison of the results, the chosen parameters will be the ones of the *Case 15*: two hidden layers, 15 hidden neurons each, Bayesian Regularization algorithm. Note that if the higher part of the OF space was more or less well reproduced by every kind of training procedure, more issues have been observed in reproducing the lower part and even more the "knee" which connects the two zones, which in some cases was totally mismatched with respect to the training set. Also note that the smoothness of the shape and the absence of unexpected oscillations has been taken into account in the analysis, mainly for the quality of the Pareto points, in order to obtain a smooth and as precise as possible front.

Case 1 - 1 hidden layer - 5 hidden neurons - BR algorithm

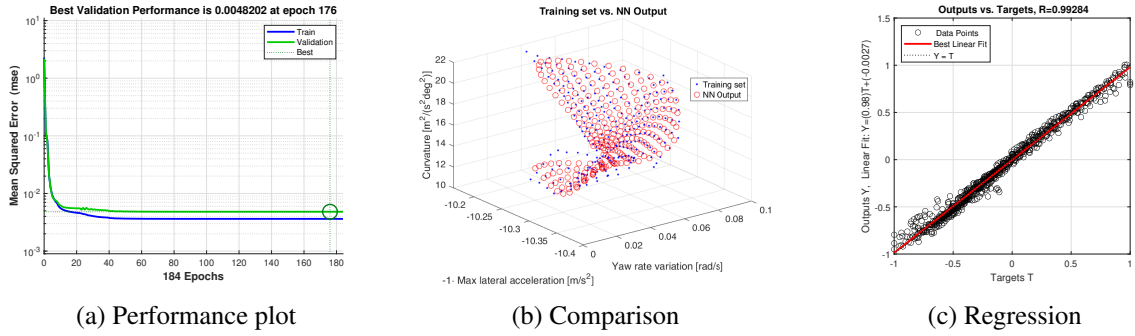


Figure 52: CASE 1

$$MSE = [3.8959 \cdot 10^{-6}, 3.4820 \cdot 10^{-5}, 0.1129]$$

Case 2 - 1 hidden layer - 5 hidden neurons - LM algorithm

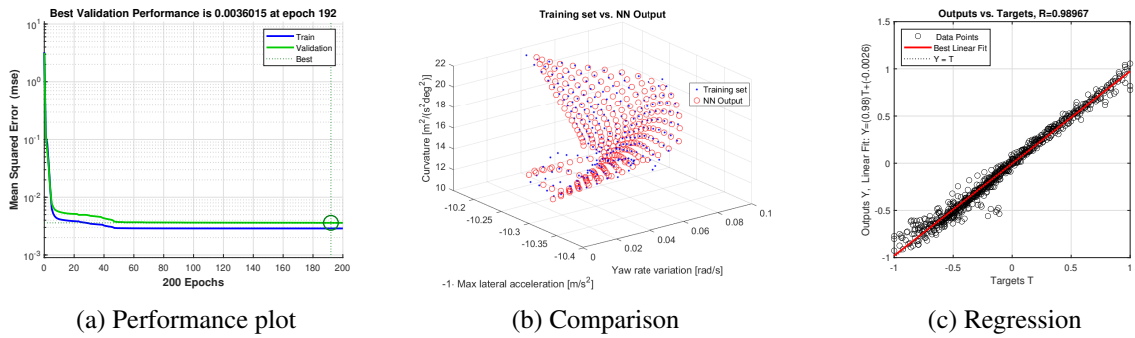


Figure 53: CASE 2

$$MSE = [2.9572 \cdot 10^{-6}, 4.3892 \cdot 10^{-5}, 0.1100]$$

Case 3 - 1 hidden layer - 10 hidden neurons - BR algorithm

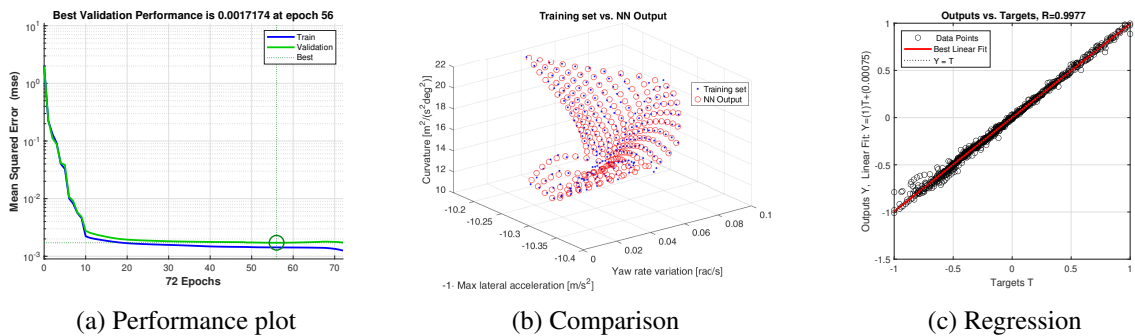


Figure 54: CASE 3

$$MSE = [7.2784 \cdot 10^{-7}, 2.3907 \cdot 10^{-5}, 0.0220]$$

Case 4 - 1 hidden layer - 10 hidden neurons - LM algorithm

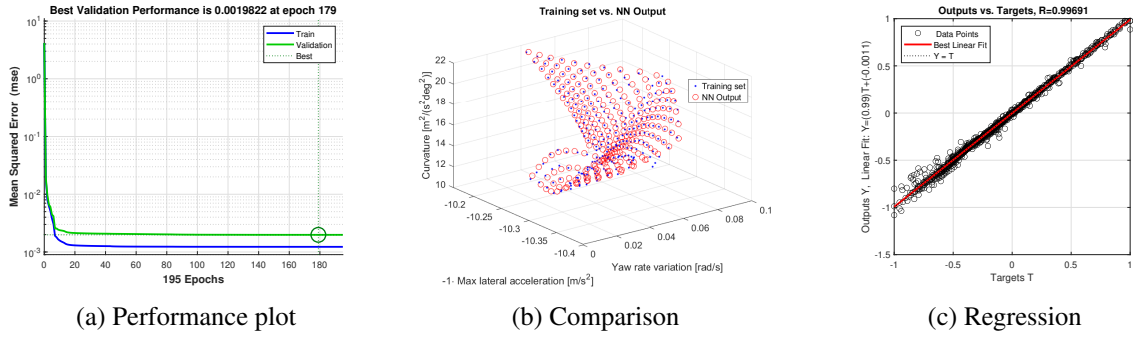


Figure 55: CASE 4

$$MSE = [6.9863 \cdot 10^{-7}, 2.5168 \cdot 10^{-5}, 0.0298]$$

Case 5 - 1 hidden layer - 20 hidden neurons - BR algorithm

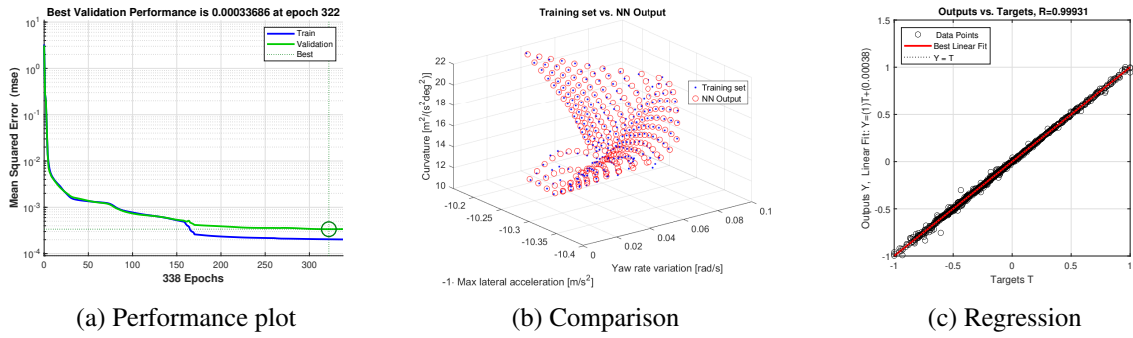


Figure 56: CASE 5

$$MSE = [1.4764 \cdot 10^{-7}, 1.5576 \cdot 10^{-6}, 0.0100]$$

Case 6 - 1 hidden layer - 20 hidden neurons - LM algorithm

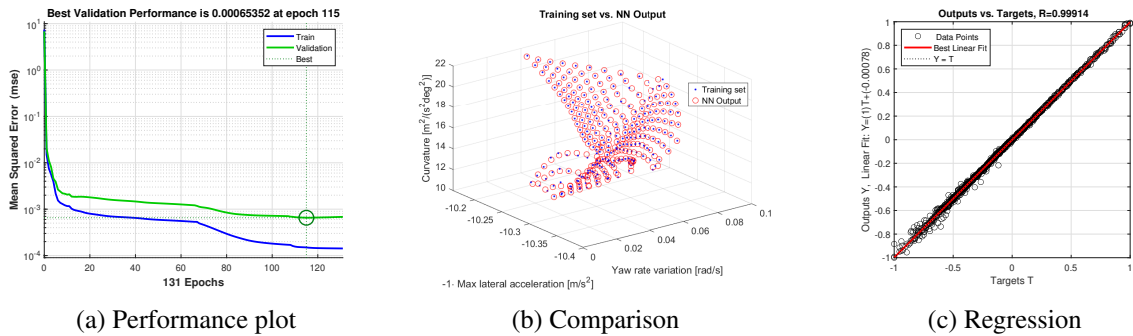


Figure 57: CASE 6

$$MSE = [2.0976 \cdot 10^{-7}, 6.2183 \cdot 10^{-6}, 0.0098]$$

Case 7 - 2 hidden layers - 3+4 hidden neurons - BR algorithm

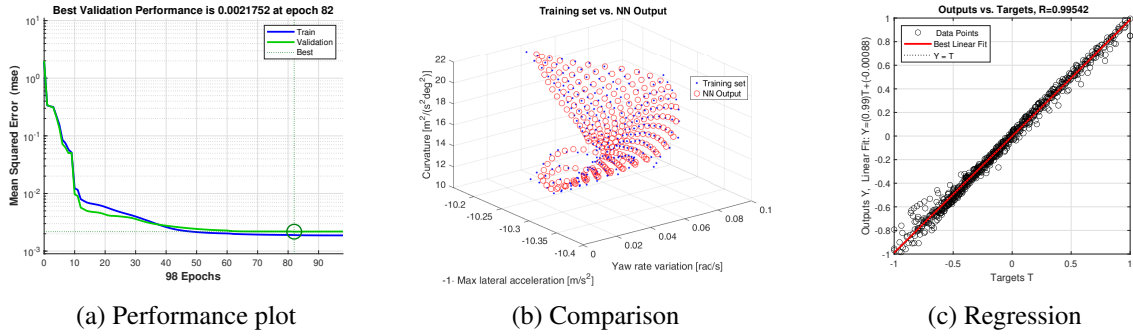


Figure 58: CASE 7

$$MSE = [2.6112 \cdot 10^{-6}, 2.8231 \cdot 10^{-5}, 0.0325]$$

Case 8 - 2 hidden layers - 3+4 hidden neurons - LM algorithm

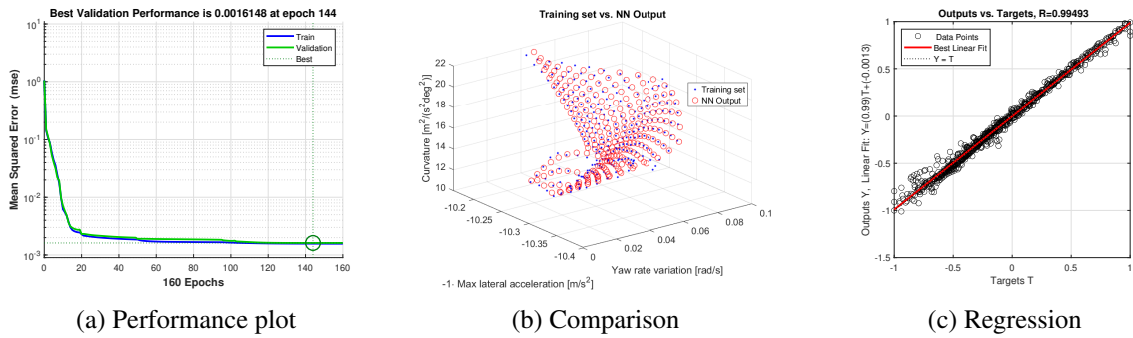


Figure 59: CASE 8

$$MSE = [1.4669 \cdot 10^{-6}, 2.9052 \cdot 10^{-5}, 0.0233]$$

Case 9 - 2 hidden layers - 5+5 hidden neurons - BR algorithm

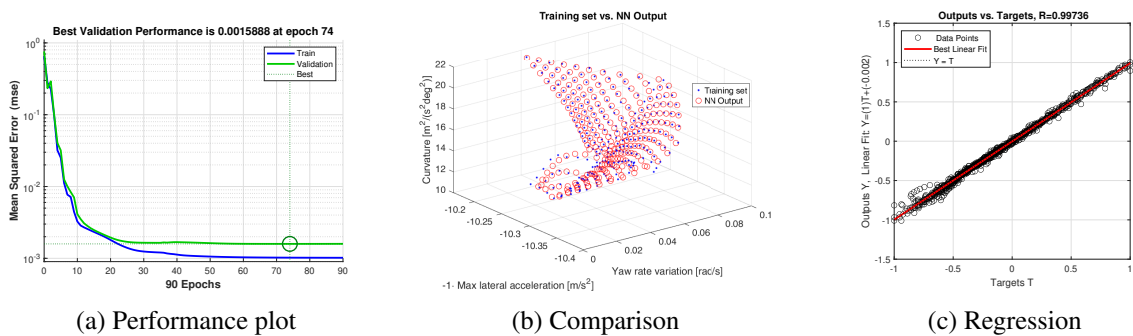


Figure 60: CASE 9

$$MSE = [4.3489 \cdot 10^{-7}, 2.5171 \cdot 10^{-5}, 0.0174]$$

Case 10 - 2 hidden layers - 5+5 hidden neurons - LM algorithm

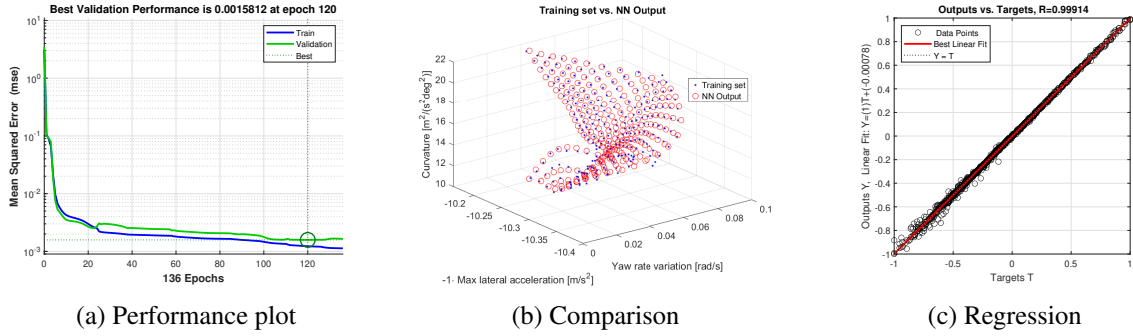


Figure 61: CASE 10

$$MSE = [1.1790 \cdot 10^{-6}, 2.2390 \cdot 10^{-5}, 0.0226]$$

Case 11 - 2 hidden layers - 10+10 hidden neurons - BR algorithm

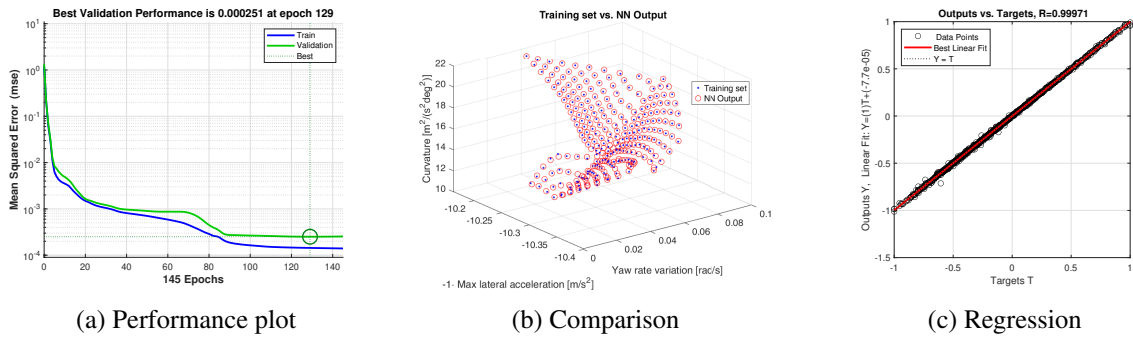


Figure 62: CASE 11

$$MSE = [1.3077 \cdot 10^{-7}, 6.0051 \cdot 10^{-7}, 0.0076]$$

Case 12 - 2 hidden layers - 10+10 hidden neurons - LM algorithm

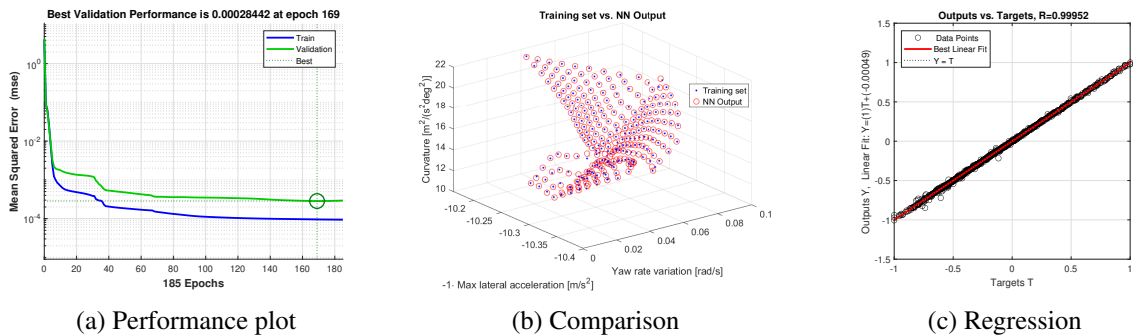


Figure 63: CASE 12

$$MSE = [8.2256 \cdot 10^{-8}, 1.1042 \cdot 10^{-6}, 0.0054]$$

Case 13 - 2 hidden layers - 12+10 hidden neurons - BR algorithm

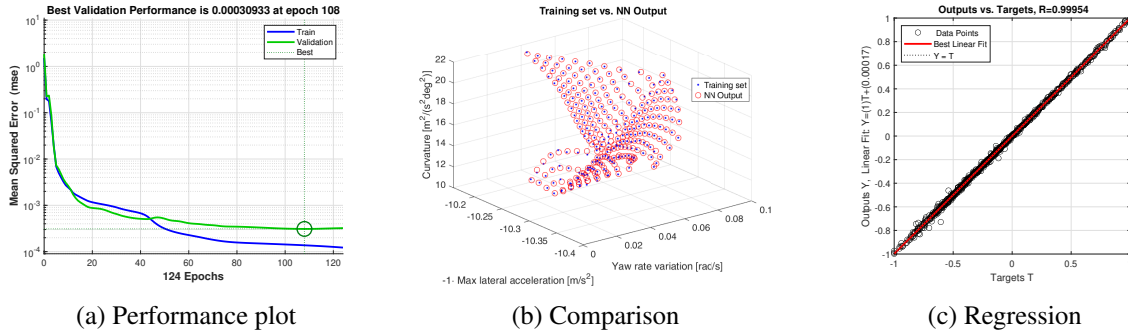


Figure 64: CASE 13

$$MSE = [1.5808 \cdot 10^{-7}, 5.5628 \cdot 10^{-7}, 0.0080]$$

Case 14 - 2 hidden layers - 12+10 hidden neurons - LM algorithm

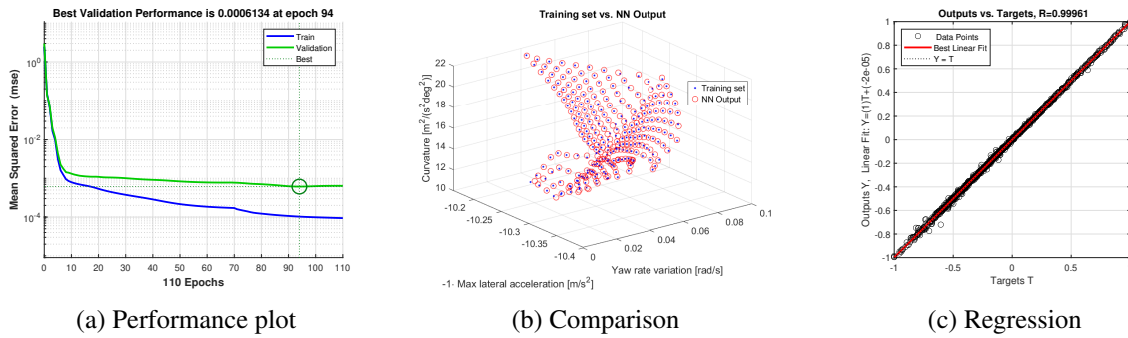


Figure 65: CASE 14

$$MSE = [1.5812 \cdot 10^{-7}, 1.0549 \cdot 10^{-6}, 0.0092]$$

Case 15 - 2 hidden layers - 15+15 hidden neurons - BR algorithm

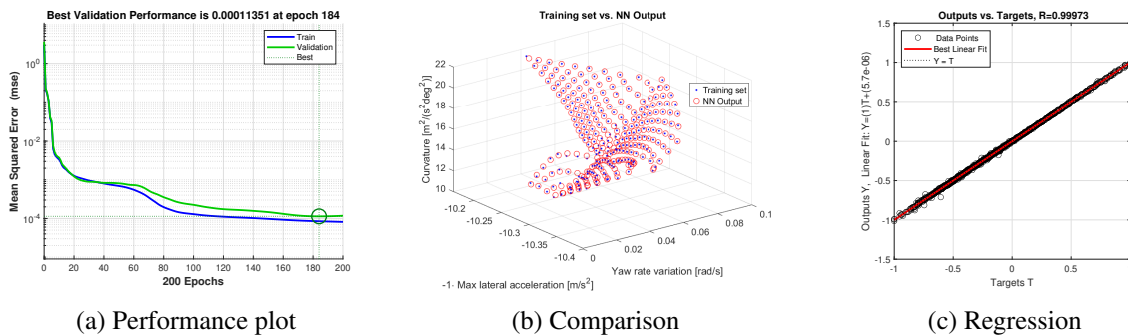


Figure 66: CASE 15

$$MSE = [4.4547 \cdot 10^{-8}, 4.0632 \cdot 10^{-7}, 0.0046]$$

Case 16 - 2 hidden layers - 15+15 hidden neurons - LM algorithm

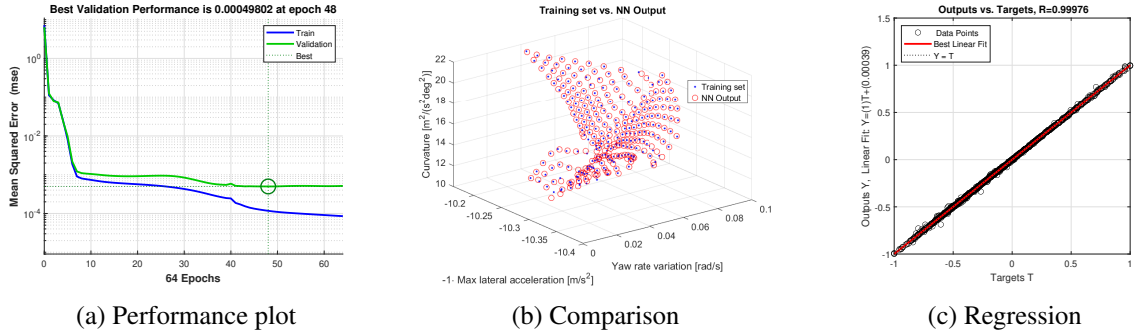


Figure 67: CASE 16

$$MSE = [1.2373 \cdot 10^{-7}, 1.3519 \cdot 10^{-6}, 0.0083]$$

Case 17 - 2 hidden layers - 20+20 hidden neurons - BR algorithm

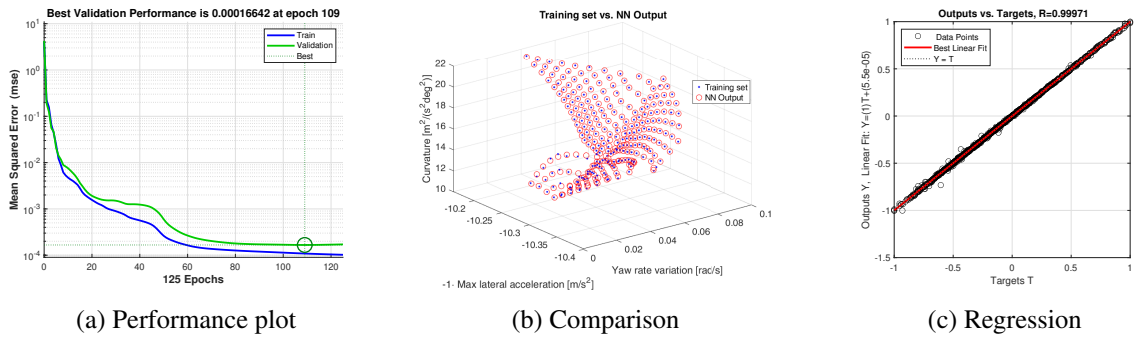


Figure 68: CASE 17

$$MSE = [2.6331 \cdot 10^{-8}, 5.5975 \cdot 10^{-7}, 0.0066]$$

Case 18 - 2 hidden layers - 20+20 hidden neurons - LM algorithm

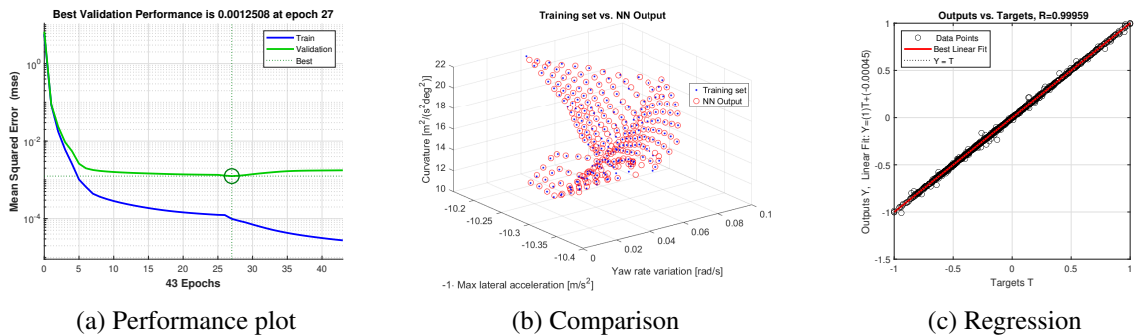


Figure 69: CASE 18

$$MSE = [2.0976 \cdot 10^{-7}, 6.2183 \cdot 10^{-6}, 0.0098]$$

8.2.3 Request 5 - Pareto front

The chosen Neural Network was Case 15, so two layers of 15 neurons each, using the Bayesian Regularization training method, since it was one of the combinations that resulted in the lowest MSE vector and one of the best regressions. It was chosen to use two hidden layers since this training system is more suited to tackle highly non linear problems. Moreover, since the most critical regions (lower part and knee, as previously described) are also the ones where the actual Pareto front is located, it is important to represent them in the most possible precise way.

After using the chosen NN, with a refined set of DV, the three-dimensional Pareto front has been computed, with the methodology presented in (Sec. 2.3.3).

10000 points are given to the network as input, corresponding to the combination of 100 values of a with as many values of H , both defined in the respective domains.

For sake of completeness, a comparison between the training set and the 10000 computed points is presented in (Fig. 70).

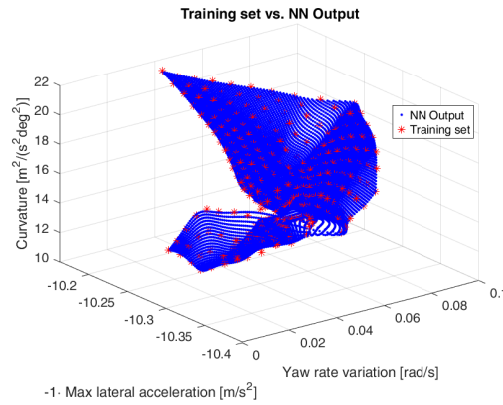
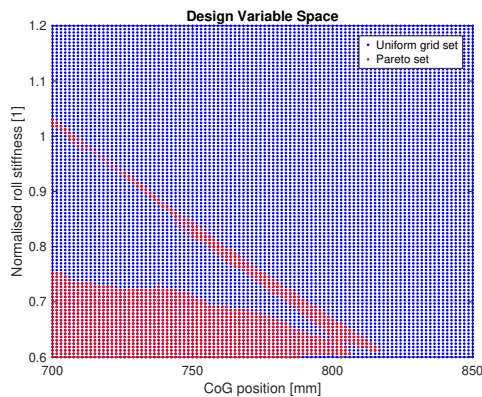
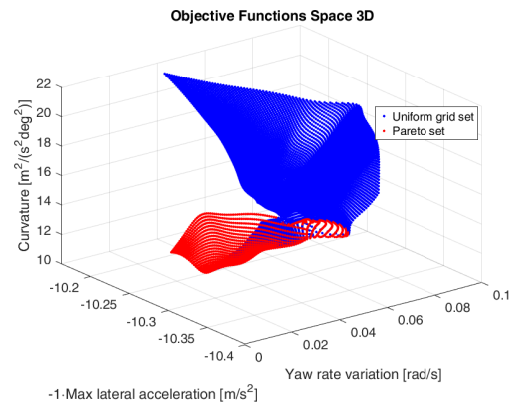


Figure 70: Training set vs. refined output

The final results are plotted in (Fig. 71).



(a) DV space



(b) OF space

Figure 71: Pareto Front

8.3 Addendum - VI-CarRealTime

To confirm the obtained results, three different solutions were extracted from the Pareto set, corresponding to the minima of the three objective functions (to better see the differences), and visualized in *VI-CarRealTime*, whose resulting plots are reported in the following. The chosen points are:

	a [mm]	H [-]	Yaw rate variation	Max lateral acceleration	Max curvature
Solution 1 (min y.r. var.)	700	0.6	0.0102	-10.2735	12.9192
Solution 2 (min a_y)	804.5455	0.6545	0.0514	-10.3722	15.0180
Solution 3 (min curvature)	700	0.7273	0.0183	-10.2939	11.6636

Table 14: Chosen Pareto points

For each solution, the handling diagram, the trajectory of the throttle release manoeuvre and the yaw-rate vs. time diagram are plotted in following figures.

8.3.1 Solution 1 - Minimum yaw rate variation

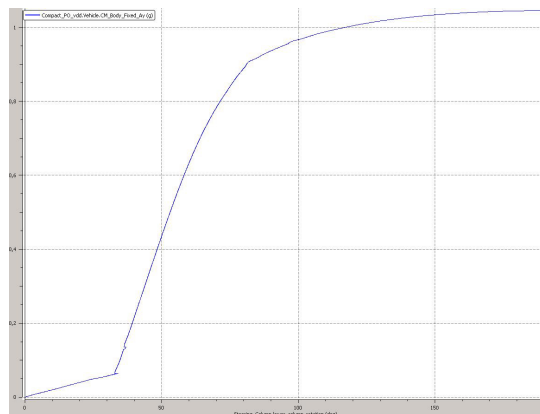
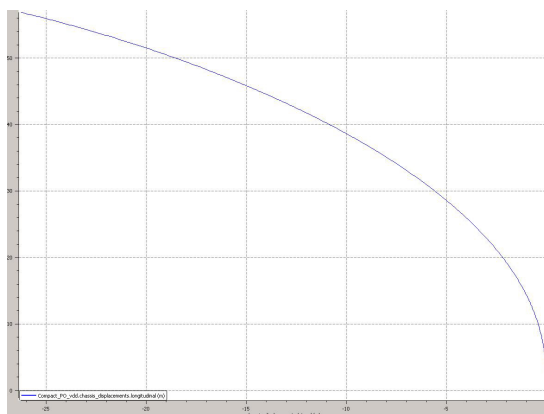
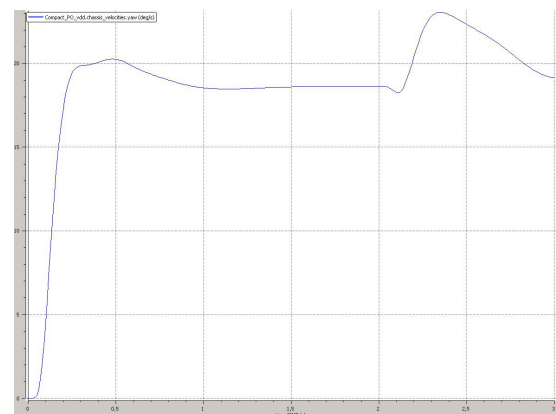


Figure 72: Handling Diagram



(a) Trajectory



(b) Yaw Rate plot

Figure 73: Throttle Release

8.3.2 Solution 2 - Maximum lateral acceleration

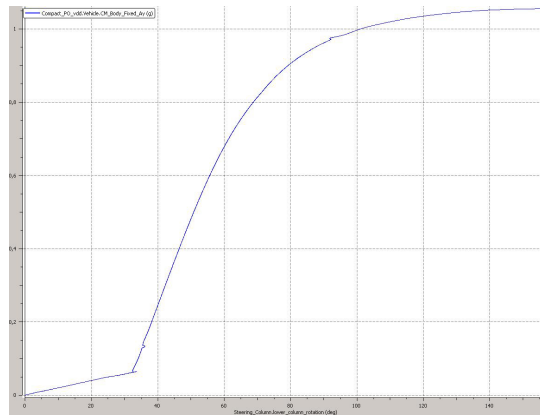
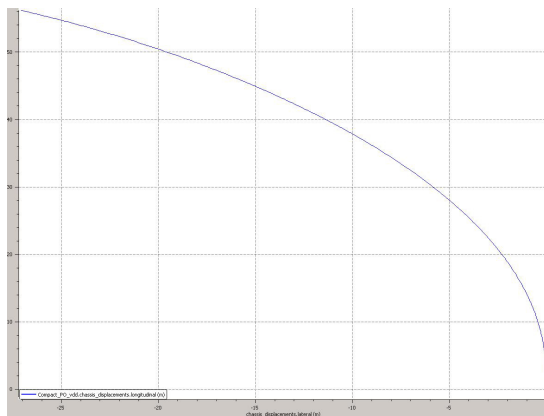
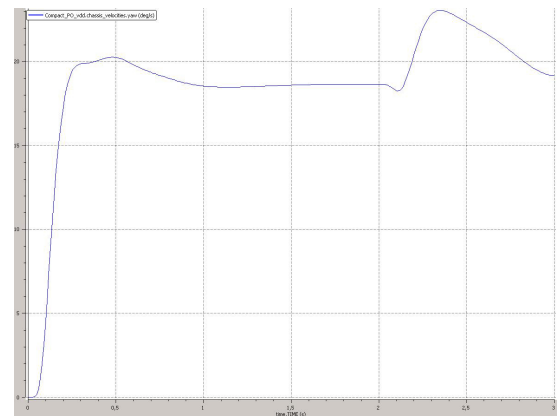


Figure 74: Handling Diagram



(a) Trajectory



(b) Yaw Rate plot

Figure 75: Throttle Release

8.3.3 Solution 3 - Minimum curvature

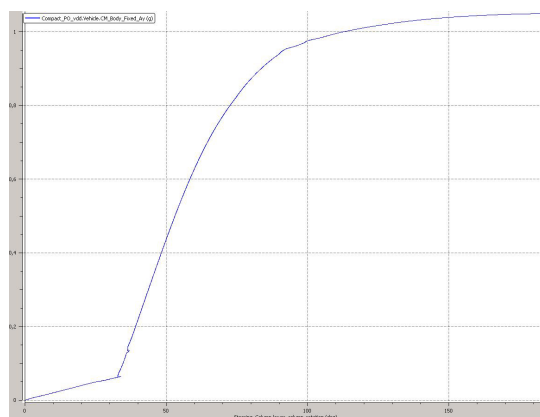
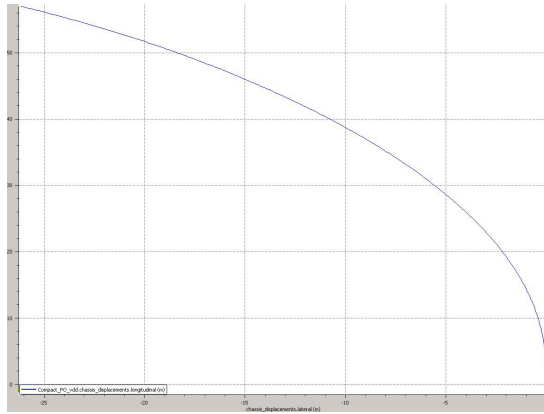
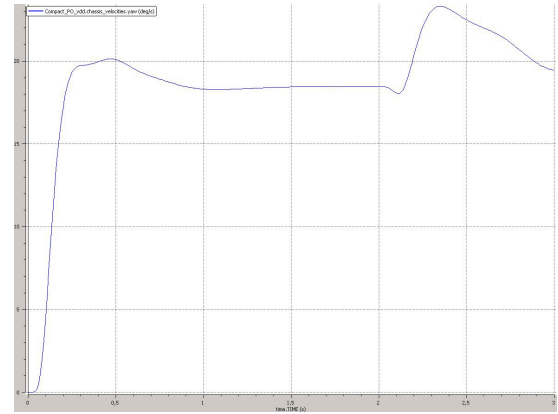


Figure 76: Handling Diagram



(a) Trajectory



(b) Yaw Rate plot

Figure 77: Throttle Release

It can be observed that, the minima of the objective functions result as expected and being them all Pareto solutions, whenever a function of the three is minimized, at least another one worsens.

Note that, as expected from the (Tab. 14), the first and the third solutions are similar each other and may be subjected to uncertainties related to the approximation procedure.

9 Lab.9 - Structural optimization

9.1 Problem definition

Structural optimization is an important task, especially for lightweight design. With this kind of analysis, the designer is able to obtain new structures which are optimized based on different objectives, like minimum mass, minimum compliance (maximum stiffness) or others.

Differently from the previous laboratories, in this case the design variables space is defined by the mass density of some elementary cells to be defined inside the space domain of the structure itself. The main idea is to remove, starting from the fully filled domain, at least a certain (predefined) amount of mass in specific locations such that the final structure will be mass-reduced and optimized according to the chosen criteria.

In the general case, the Design Variable is the density of each single cell belonging to the discretized space domain. Without any kind of intervention, the optimal solution could result in some cells being defined by a (normalized) density in the middle between 0 (no mass) and 1 (fully filled with the chosen material), which is clearly an unfeasible solution.

To face this problem, specific parameters will be introduced to avoid intermediate density values and obtain a totally black and white solution (where black means mass and white means no mass).

In any case, two constraints concerning structural safety are added to the problem:

1. No yielding (plastic deformation)
2. No buckling (elastic instability)

In our case, the aim is the minimization of the compliance, meaning that the solution will belong to the Pareto-optimal set and will be the one that maximize the stiffness, for a given minimum amount of mass reduction (volume reduction with fixed density). With this formulation, we are able to obtain a single final solution.

Two methodologies are proposed for this laboratory, to solve two different case studies:

1. MatLab function `topology.m` corresponding to the *99 line topology optimization code* by Ole Sigmund.
2. by means of the *Altair Inspire* software.

In the following sections, the results of both the approaches are provided.

9.2 99 line topology optimization code

9.2.1 Function definition

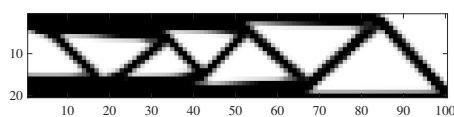
In the provided `topology.m` function, the following parameters need to be defined:

1. number of cells in x direction;
2. number of cells in y direction;
3. penalization value;
4. volume fraction to be removed from the initial domain;
5. the filter radius.

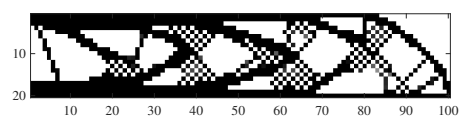
In particular, the penalization value is the parameter which allows to obtain the black and white results explained in (Sec. 9.1), through the intervention of a (non-linear) exponential function with exponent equal to the penalization itself. By setting it to 1, no penalization is introduced and the solution will accept intermediate density values. By setting it to higher values, a certain level of penalization will be introduced to cells containing intermediate density values: a value of 3 will result in a almost totally black and white solution.

The penalization acts as a kind of constraint which will make the solution change, worsening it (in the optimization point of view), but making it realisable.

Moreover, the filter radius is a parameter which is introduced to avoid a chequerboard solution (by performing a weighted average taking into account the neighbouring cells densities), as shown in (Fig. 78), and to obtain the mesh independency property. Using this parameter, the system is forced to place in the neighbouring cells similar values of density, creating smooth and continuous regions of material.



(a) Filter radius: 1.15 - smoother



(b) Filter radius: 0.5 - chequerboard

Figure 78: Filter radius effect

Obviously, the result of (Fig. 78b), is not a feasible nor useful solution.

The mesh independency property is analysed in the following section.

9.2.2 Request 1: Mesh independency

As anticipated, the filter radius is also required to ensure the mesh independency property in the optimization problem. If such property is obtained, the final solution will be the same for different number of cells in x and y direction. To this purpose, the filter radius needs to be incremented proportionally to the number of cells in x and y direction (i.e. a doubled number of cells will correspond to a doubled filter radius).

In the following, four different cases are compared.

Case 0 - Baseline

x cells	y cells	Vol. fract. to remove	Pen. value	Filter radius
100	20	0.5	3	1.15

$$clearance = 728.4168 \text{ rad/m}$$

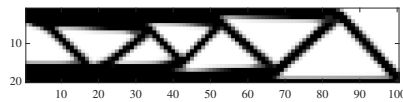


Figure 79: Baseline solution

Case 1 - Doubled cells

x cells	y cells	Vol. fract. to remove	Pen. value	Filter radius
200	40	0.5	3	2.3

$$clearance = 733.0263 \text{ rad/m}$$

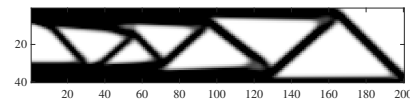


Figure 80: Doubled cells solution

Case 2 - Tripled cells

x cells	y cells	Vol. fract. to remove	Pen. value	Filter radius
300	60	0.5	3	3.45

$$clearance = 731.1331 \text{ rad/m}$$

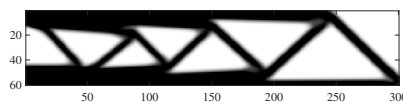


Figure 81: Tripled cells solution

Case 3 - x0.85 cells

x cells	y cells	Vol. fract. to remove	Pen. value	Filter radius
85	17	0.5	3	0.9775

$$clearance = 714.2117 \text{ rad/m}$$

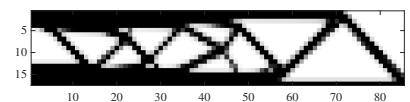


Figure 82: x0.85 cells parameters

Note that a reduction with a coefficient lower than 0.85 has been associated to convergence difficulties.

It can be observed that, apart from accuracy differences related of the mesh resolution and thus quality of the results, all the obtained structures are practically the same and so the associated clearance.

The only slightly different solution is the case 3, where probably a too coarse mesh leads to a non precise clearance computing. Also the graphical solution is just similar to the previous one, with a slightly less precise and well defined solution.

The main difference within the cases has been observed in the required computational time (much larger with increasing number of cells).

In the next section, the effects of the penalization parameter are investigated.

9.2.3 Request 2: Penalization effects

As saw in (Sec. 9.1), the penalization acts as a constraint for our optimal solution. For this reason, we expect that the solutions without penalization (parameter equal to 1) correspond to a lower value of the clearance.

Cases 0 and 1 of (Sec. 9.2.2) are now presented in the case of *Penalization value* = 1.

Case 0

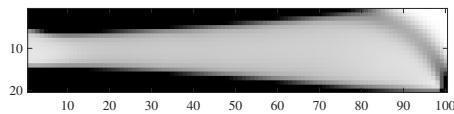


Figure 83: Baseline with no penalization

$$clearance = 631.9010 \text{ rad/m}$$

Case 1

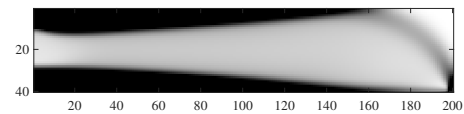


Figure 84: Doubled cells with no penalization

$$clearance = 643.3970 \text{ rad/m}$$

Also in these cases, the two results are practically the same, with a clearance reduction of 96.5158 rad/m for case 0 and of 98.6293 rad/m for case 1.

It is here clear how the density tends in both cases to assume values which are intermediated between a fully occupied cell and a totally empty one, a situation which could be practically realized only by means of special composite materials.

In the next section, the second approach is presented.

9.3 Altair Inspire

9.3.1 Problem setup

A second structural optimization problem was tackled, using the software Altair Inspire. The objective was to maximize the stiffness, while simultaneously reducing the arm mass of a MacPherson suspension lower control arm. The starting point of the optimization is presented in (Fig. 85), where the control arm has the shape of a parallelepiped: the gray coloured parts are frozen parts, meaning that the solver will not remove material there since they correspond to the important connection points of the LCA to the hub carrier and chassis; while the brown part is where the solver can remove material to reach the optimization goal.

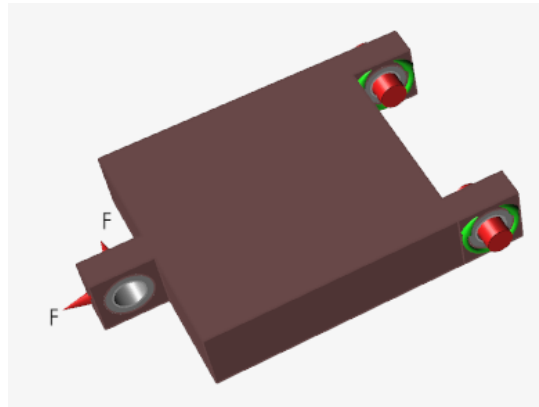


Figure 85: Initial domain

The LCA was subject to forces coming from the tyre, which resulted in a longitudinal component, along x, and a lateral one, along y: forces along the vertical direction were neglected. To compute the forces mentioned above, the selected vehicle was a Citroen C3 PureTech 110 2024, which had an unladen mass of 1210 kg. Performing simple calculations based on the vertical force that the car applies to the ground, the following forces acting on the LCA were calculated.

$$\begin{cases} F_x = F_{z,st} * 2.5 * 1.2 = 8903 N \\ F_y = F_{z,st} * 2.5 * 0.7 = 5193 N \end{cases}$$

The problem was set up as follows:

- Material: Aluminium 6061 T3
- Boundary Conditions: two hinges in correspondence of the bushings, allowing rotations around x
- Objective: Maximum stiffness
- Mass reduction: At least 30 %
- Minimum thickness: 15 mm
- Manufacturing technique: Stamping with form extraction at LCA midplane

9.3.2 Results

The following result (Fig. 86) was obtained with the simulation stated above: the solver completely removed the sharp edges near the connection to the hub carrier, while in the middle of the LCA it removed almost all the available material, leaving a cross-shaped reinforcement structure in the middle of the two thick edges. This cross-shaped reinforcement is useful to support the loads that come from both the longitudinal and lateral directions. Moreover, the arm in the opposite direction with respect to the longitudinal force (the one working in traction) is thicker, allowing for maximum stiffness while decreasing the overall mass from 7.775 kg to 3.557 kg , corresponding to a 54.25% mass reduction (well above the target).

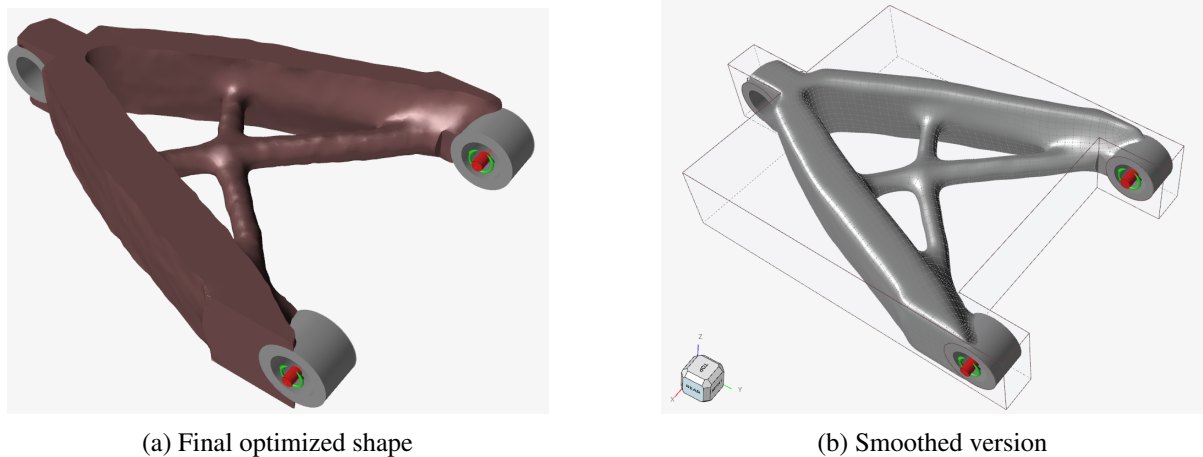


Figure 86: Optimization results (max stiffness)

Here below are presented two static deformation cases, before and after the optimization. It can be observed that the maximum displacement increases of less than one order of magnitude due to the mass reduction: the maximum value for the initial domain is about 0.092 mm , the final about 0.14 mm .

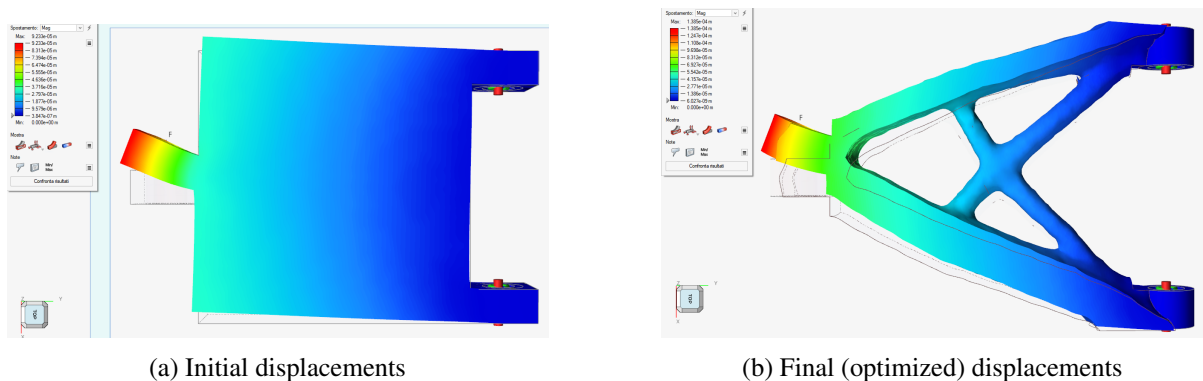


Figure 87: Displacements comparison

Finally, the Von Mises stresses are shown in the following figure. The maximum value, at the connection to the hub carrier, is about 43.6 MPa , well below the yielding stress (305 MPa).

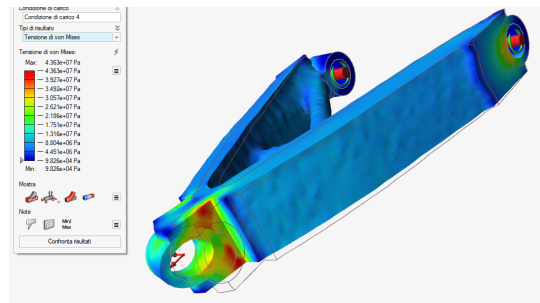


Figure 88: Stresses - optimized